

CS570
Analysis of Algorithms
Fall 2016
Exam II

Name: _____

Student ID: _____

Email Address: _____

_____ **Check if DEN Student**

	Maximum	Received
Problem 1	20	
Problem 2	16	
Problem 3	16	
Problem 4	16	
Problem 5	16	
Problem 6	16	
Total	100	

Instructions:

1. This is a 2-hr exam. Closed book and notes
 2. If a description to an algorithm or a proof is required please limit your description or proof to within 150 words, preferably not exceeding the space allotted for that question.
 3. No space other than the pages in the exam booklet will be scanned for grading.
 4. If you require an additional page for a question, you can use the extra page provided within this booklet. However please indicate clearly that you are continuing the solution on the additional page.
- 1) 20 pts
Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[FALSE] The run-time is $O(nW)$ so it's pseudopolynomial

0-1 knapsack problem can be solved using dynamic programming in polynomial time

[FALSE] If neither of the two nodes are reachable by the negative cycle, the distance between them is calculated accurately.

The Bellman-Ford algorithm always fails to find the shortest path between two nodes in a graph if there is a negative cycle present in the graph.

[TRUE] Value of max flow = capacity of min-cut (iterate over all edges of the min-cut)

Given the min-cut, we can find the value of max flow in $O(|E|)$.

[TRUE] Set the penalties to 0 and reward of 1 for every match

The sequence alignment algorithm can be used to find the longest common subsequence between two given sequences.

[FALSE] You only have to do it once.

In dynamic programming you must calculate the optimal value of a sub-problem twice, once during the bottom up pass and once during the top down pass.

[TRUE] For any cut which is not a min-cut this is True.

Maximum value of an s-t flow could be less than the capacity of a given s-t cut in a flow network.

[FALSE] Consider a graph with 4 nodes s, a, b and t and $E = \{(s,a), (a,t), (s,b), (b,t)\}$. The increase in the flow would be $f + 2$.

Suppose the maximum (s, t)-flow of some graph has value f . Now we increase the capacity of every edge by 1. Then the maximum (s, t)-flow in this modified graph will have value at most $f + 1$.

[FALSE] The Orlin algorithm runs in $O(mn)$

There are no known polynomial-time algorithms to solve maximum flow.

[FALSE] Edges having capacity 1 does not mean $|f| = 1$ so the Ford-Fulkerson is still going to be pseudo-polynomial.

If all edges in a graph have capacity 1, then Ford-Fulkerson runs in linear time.

[FALSE] Choice of augmenting path can still affect the number of iterations of the inner loop.

In the scaled version of the Ford Fulkerson algorithm, choice of augmenting paths cannot affect the number of iterations.

2) 16 pts

Given $N(G(V, E), s, t, c)$, a flow network with source s , sink t , and positive edge capacities c , we are asked to reduce the max flow by removing at most k edges. Prove or disprove the following statement:

Maximum reduction in flow can be achieved by finding a min cut in N , creating a sorted list of the edges going out of this cut in decreasing order and then removing the top k edges in this list.

The Statement is False. (8 points)

Proof (8 points). This requires a counter example where deleting the edges from the min-cut would not result in maximum reduction in max flow.

Many students tried to prove this statement. The statement is true only if all the edges have equal capacities. If the proof was correct based on that assumption, you were given 8 points.

3) 16 pts

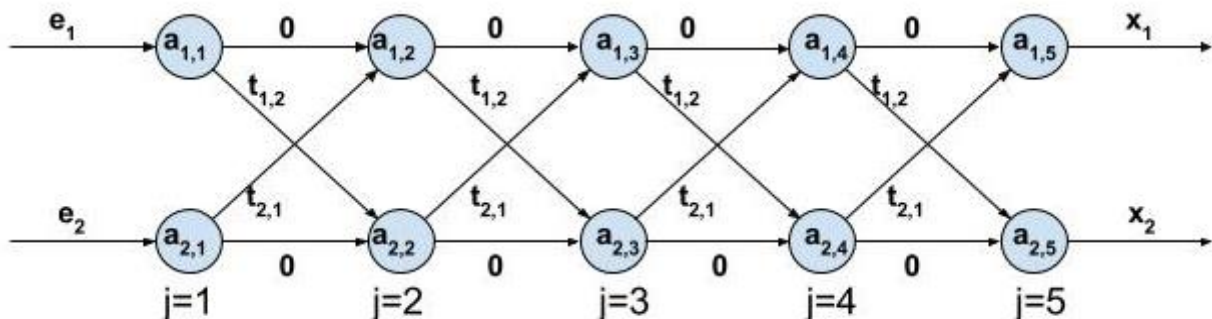
A car factory has k assembly lines, each with n stations. A station is denoted by $S_{i,j}$ where i indicates that the station is on the i -th assembly line ($0 < i \leq k$), and j indicates the station is the j -th station along the assembly line ($0 < j \leq n$). Each station is dedicated to some sort of work like engine fitting, body fitting, painting and so on. So, a car chassis must pass through each of the n stations in that order before exiting the factory. The time taken per station is denoted by $a_{i,j}$. Parallel stations of the k assembly lines perform the same task. After the car passes through station $S_{i,j}$, it will continue to station $S_{i,j+1}$ unless we decide to transfer it to another line. Continuing on the same line incurs no extra cost, but transferring from line x at station $j-1$ to line y at station j takes time $t_{x,y}$. Each assembly line takes an entry time e_i and exit time x_i which may be different for each of these k lines. Give an algorithm for computing the minimum time it will take to build a car chassis.

Below figure provides one example to explain the problem. The example shows $k=2$ assembly lines and 5 stations per line. To illustrate how to evaluate the cost, let's consider two cases: If we always use assembly line 1, the time cost will be:

$$e_1 + a_{1,1} + a_{1,2} + a_{1,3} + a_{1,4} + a_{1,5} + x_1.$$

If we use stations $s_{1,1} \rightarrow s_{2,2} \rightarrow s_{1,3} \rightarrow s_{2,4} \rightarrow s_{2,5}$, the time cost will be

$$e_1 + a_{1,1} + t_{1,2} + a_{2,2} + t_{2,1} + a_{1,3} + t_{1,2} + a_{2,4} + a_{2,5} + x_2.$$



- a) Recursively define the value of an optimal solution (recurrence formula) (6 pts)

Dynamic programming Solution

Subproblem: $\text{minCost}(i,j)$, if we use assembly line- i at station- j , the minimal cost so far.

Initialization: $\text{minCost}(1,1) = e_1 + a_{1,1}$, $\text{minCost}(2,1) = e_2 + a_{2,1}$, ... $\text{minCost}(k,1) = e_k + a_{k,1}$ (1 point)

Recurrence relation: $\text{minCost}(i,j) = \min_{(1 \leq l \leq k)} \text{minCost}(l,j-1) + t_{l,i} + a_{i,j}$ (4 points)
($t_{i,i} = 0$ in this formulation)

Final cost: $\text{minCost}(i, \text{exit}) = \text{minCost}(i, n) + x_i$ (1 point)

Attention: Some students may assume that the car can only pass to the station (i, j) from adjacent lines $i-1$ and $i+1$. In this case, you will have 3 points off.

- b) Describe (using pseudocode) how the value of an optimal solution is obtained using iteration. Make sure you include any initialization required. (6 pts)

// initialization, boundary condition

For $i=1:k$

$\text{minCost}[i][1] = e[i] + a[i][1]$ (1 point)

// recurrence

For $t=2:n$

For $i=1:k$

$\text{minCost}[i][t] = \text{minCost}[1][t-1] + t[1][i] + a[i][t]$

for $j=2:k$

$\text{minCost}[i][t] = \min(\text{minCost}[j][t-1] + t[j][i] + a[i][t], \text{minCost}[i][t])$ (4 points)

for $i=1:k$

$\text{minCost}[i][n] = \text{minCost}[i][n] + x[i]$

// final solution

return $\min(\text{minCost}[1][n], \text{minCost}[2][n], \dots, \text{minCost}[k][n])$ (1 point)

Attention: some students may ignore the initialization.

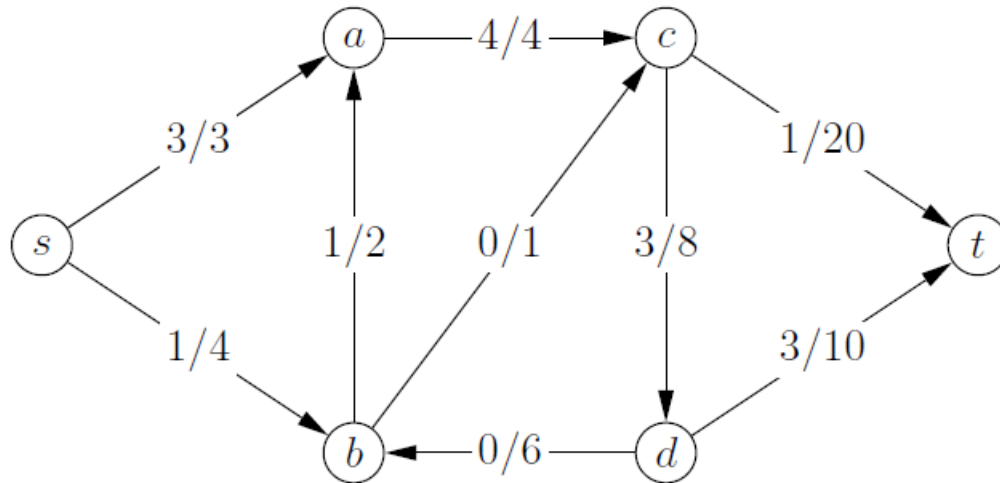
- c) What is the complexity of your solution in part b? (4 pts)

From the recurrence loops, we can tell the complexity is $O(n \times (k^2))$

Attention: If you don't show your work process and your final solution is incorrect, you will have 4 points off.

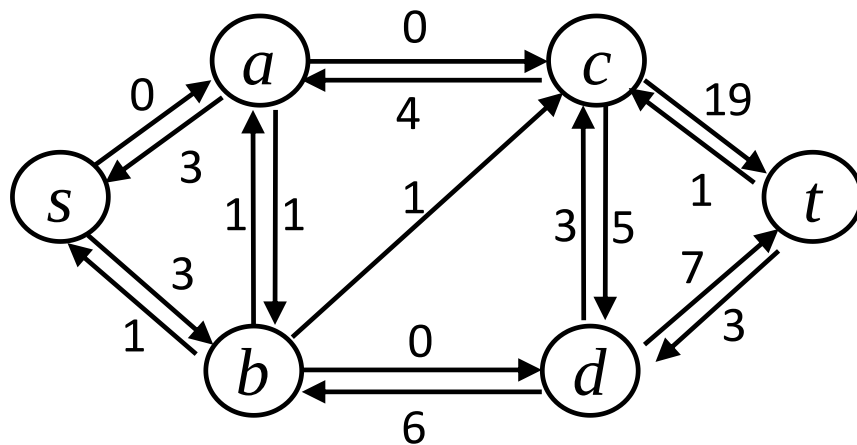
4) 16 pts

You are given a directed graph which after a few iterations of Ford-Fulkerson has the following flow. The labeling of edges indicate flow/capacity:



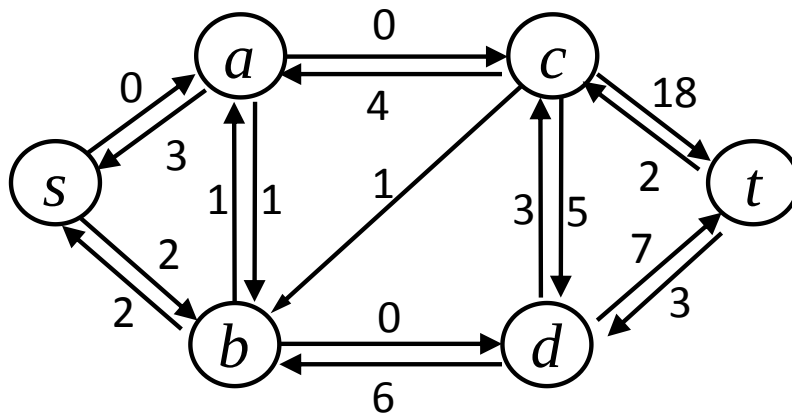
a) Draw the corresponding residual graph. (5 pts)

Each mistake in edge direction or value would cause one-point deduction.



b) Is this a max flow? If yes, indicate why. If no, find max flow. (6 pts)

No, since there is still path like $s-b-c-t$ that can be augmented with flow 1 (3 points) The max flow is $4+1=5$ that can be obtained from the residual graph shown below (3 points).



c) What is the min-cut? (5 pts)

Min-cut: $(\{s, a, b\}, \{c, d, t\})$ (4 points). For the min-cut you need to show it on the graph or write it in the form (A, B) where A, B are the two parts of the original graph. The value of min-cut equals to $C_{ac} + C_{bc} = 4 + 1 = 5$. (1 point)

Note that the other cuts like $(\{s, b\}, \{a, c, d, t\})$ with value of 6 etc., have larger values than the min-cut with value of 5 and cannot be considered as min-cut for the network.

5) 16 pts

Given a rod of length n inches and an array of prices that contains prices of all pieces of size smaller than or equal to n . Determine the maximum value obtainable by cutting up the rod and selling the pieces. For example, if the length of the rod is 8 and the values of different pieces are given as in the following table, then the maximum obtainable value is 23 (by cutting the rod into two pieces of lengths 2 and 6). Notation: $\text{Price}(i)$ is the price of a rod of length i

Length	1	2	3	4	5	6	7	8
Price	1	5	8	9	10	18	17	20

And if the prices are as given in the following table, then the maximum obtainable value is 24 (by cutting the rod into eight pieces of length 1)

Length	1	2	3	4	5	6	7	8
Price	3	5	8	9	10	19	17	20

a) Recursively define the value of an optimal solution (recurrence formula) (6 pts)

The following four solutions are correct –

Soln 1:

$\text{OPT}(i)$: best possible price obtained for a rod of length i

$\text{OPT}(i) = \max\{p(j) + \text{OPT}(i-j)\}$ for all j in $\{1..i\}$

Time complexity: $O(n^2)$

Here, the assumption is that the rod needs to be cut and cannot be left uncut.

Soln 2:

$\text{OPT}(i)$: best possible price obtained for a rod of length i

$\text{OPT}(i) = \max\{\text{OPT}(j) + \text{OPT}(i-j), p[i]\}$ for all j in $\{1..i\}$

Time complexity: $O(n^2)$

Here, the assumption is that the rod need not be cut.

Soln 3:

$\text{OPT}(i, j)$: best possible price obtained for a rod of length i with j cuts

$\text{OPT}(i, j) = \max\{\text{OPT}(k, j-1) + p[i-k], \text{OPT}(i, j-1)\}$ for all k in $\{1..i\}$

Time complexity: $O(n^3)$

4 points have been deducted for high time complexity.

Many students have used the above definition for OPT (i.e. j cuts as opposed to cuts 1..j) but have not used a third variable k. This is incorrect and marks have been deducted for this since we can have multiple cuts of same length.

Soln 4:

OPT(i, j): best possible price obtained for a rod of length i when the longest cut is no longer than j

$$\text{OPT}(i, j) = \max\{\text{OPT}(i-j, j) + p[j], \text{OPT}(i, j-1)\}$$

Time complexity: $O(n^2)$

Grading rubric:

(a) Definition of OPT: 2 marks

Recurrence: 3 marks

Final answer: 1 mark

(b) Initialization: 2 marks

Loop indices: 3 marks

Return final answer: 1 marks

(c) No partial marking

Note that no marks have been given for part c if the recurrence formed in part a was incorrect.

b) Describe (using pseudocode) how the value of an optimal solution is obtained using iteration. Make sure you include any initialization required. (6 pts)

Solution 1 can be written using iteration as

```
int cutRod(int price[], int n)
{
    int val[n+1];
    val[0] = 0;
    int i, j;

    // Build the table val[] in bottom up manner and return the last entry
    // from the table
    for (i = 1; i <= n; i++)
    {
        int max_val = INT_MIN;
        for (j = 1; j <= i; j++)
            max_val = max(max_val, price[j] + val[i-j]);
    }
}
```

```
        val[i] = max_val;
    }

    return val[n];
}
// end OR

return cutRod(arr, size)
```

c) What is the complexity of your solution in part b? (4 pts)

$O(n^2)$

6) 16 pts

There are n students in a class. We want to choose a subset of k students as a committee. There has to be m_1 number of freshmen, m_2 number of sophomores, m_3 number of juniors, and m_4 number of seniors in the committee. Each student is from one of k departments, where $k = m_1 + m_2 + m_3 + m_4$. Exactly one student from each department has to be chosen for the committee. We are given a list of students, their home departments, and their class (freshman, sophomore, junior, senior).

Describe an efficient algorithm based on network flow techniques to select who should be on the committee such that the above constraints are all satisfied.

- a) Describe how to construct a flow network to solve this problem, including the description of nodes, edges, edge directions and their capacities. (10 pts)

- b) Describe how the solution to the network flow problem you described in part a corresponds to the problem of determining who should be on the committee. (6 pts)

Solution: Consider the following graph, the source is connected to K departments with capacity one. Each department is connected to a subset of n students, that belongs to the department with capacity one. Each student is connected to the respective group (freshman, sophomore, junior, senior) with capacity one. Each group is connected to the sink with capacity m_i for group i . The maximum flow in this problem finds the set of students to be included in the committee. The students' nodes that flow pass through them are shows who should be in the committee

a) The problem has two important constraints :

1. Exactly one student from each department must be selected.
 k nodes and k edges with correct capacity expected. (3 points)
2. Number of selected students from each class must be m_1 for the freshmen, ..
 4 nodes for classes with capacity m_1, m_2, m_3, m_4 expected. (3 points)

For students you can either put some nodes or some edges, But if your graph doesn't give the correct answer you will not get point just for adding n nodes for students. (2 points)

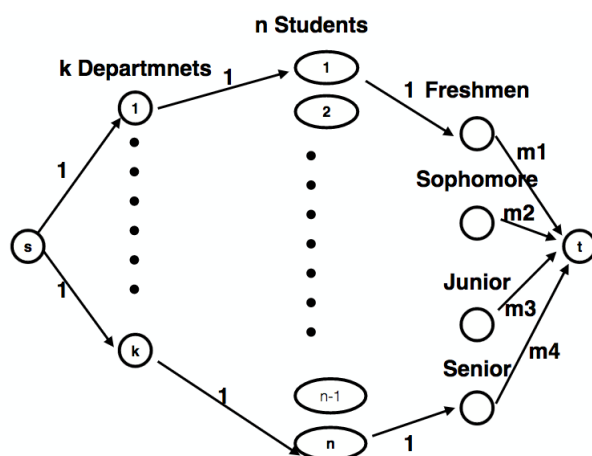
The network flow graph gives the right set of students (2 points)

b) claim: Picking k students for the committee is possible iff The max flow of the given graph in part a is equal to k . (2 points)

Why your graph satisfies the constraint of the problem (2 points)

How the set of students can be determined based on you graph (2 points)

You can either explain these three question or prove the claim, and you will get 6 points.



Additional Space

Additional Space

CS570
Analysis of Algorithms
Spring 2017
Exam II

Name: _____
Student ID: _____
Email Address: _____

_____ **Check if DEN Student**

	Maximum	Received
Problem 1	20	
Problem 2	10	
Problem 3	10	
Problem 4	15	
Problem 5	15	
Problem 6	15	
Problem 7	15	
Total		

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[/FALSE] The value of the max flow is not enough and we need the complete flow

Given the value of max flow, we can find a min-cut in linear time.

[/FALSE]

The Ford-Fulkerson algorithm can compute the maximum flow in polynomial time.

[/FALSE]

A network with unique maximum flow has a unique min-cut.

[TRUE/]

If all of the edge capacities in a graph are an integer multiple of 3, then the value of the maximum flow will be a multiple of 3.

[/FALSE]

The Floyd-Warshall algorithm always fails to find the shortest path between two nodes in a graph with a negative cycle.

[/FALSE] 0/1 knapsack does not have a polynomial time dynamic programming solution—not even a weakly polynomial solution. 0/1 knapsack has a pseudopolynomial time dynamic programming solution which is basically an exponential time solution (with respect to its input size)

0/1 knapsack problem can be solved using dynamic programming in polynomial time, but not **strongly** polynomial time.

[/FALSE]

If a dynamic programming algorithm has n subproblems, then its running time complexity is $O(n)$.

[/FALSE]

The Travelling Salesman problem can be solved using dynamic programming in polynomial time

[/FALSE]

If flow in a network has a cycle, this flow is not a valid flow.

[FALSE/] We can't do this for undirected graphs

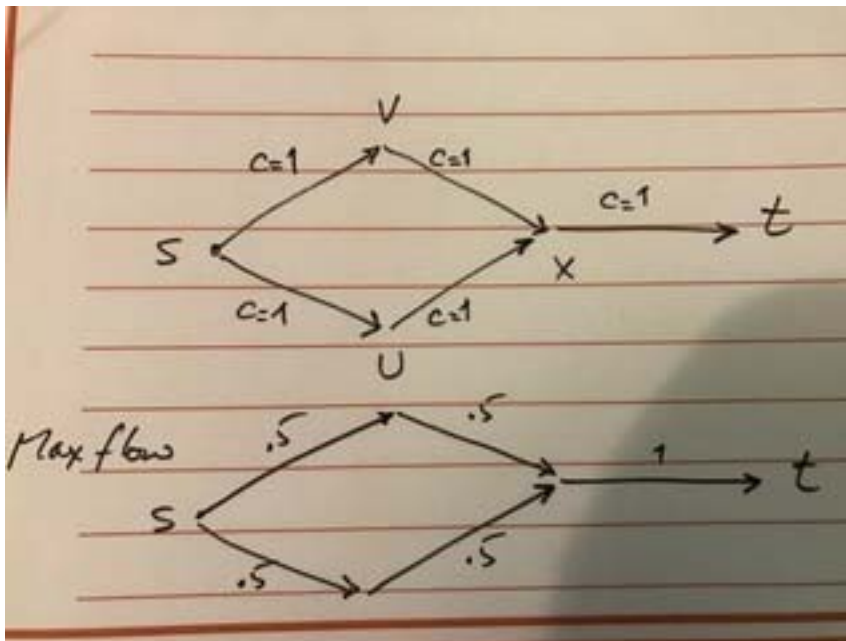
We can use the Bellman-Ford algorithm for undirected graph with negative edge weights.

2) 10 pts

Given $N(G(V, E), s, t, c)$, a flow network with source s , sink t , and positive integer edge capacities $c(e)$ for every $e \in E$. Prove or disprove the following statement:

A maximum flow in an integer capacity graph must have integral (integer) flow on each edge.

Counterexample:



Grading:

A correct counter example gets full 10 points, otherwise 0 points is given.

Common Mistake:

0 is an integer so it will not count as a non-integral flow.

3) 10 pts

The subset sum problem is defined as follows: Given a set of n positive integers $S = \{a_1, a_2, \dots, a_n\}$ and a target value T , does there exist a subset of S such that the sum of the elements in the subset is equal to T ? Let's define a boolean matrix M where $M(i, j)$ is true if there exists a subset of $\{a_1, a_2, \dots, a_i\}$ whose sum equals j . Which one of the following recurrences is valid? (circle one)

1) $M(i, j) = M(i-1, j) \cup M(i, j - a_i)$

2) $M(i, j) = M(i-1, j) \cup M(i-1, j - a_i)$ This is the correct solution

3) $M(i, j) = M(i-1, j-1) \cup M(i-1, j)$

4) $M(i, j) = M(i, j - a_i) \cup M(i-1, j - a_i)$

Solution:

For computing $M(i, j)$ you either include a_i or you don't. If you include a_i then we have $M(i, j) = M(i-1, j - a_i)$. If you don't include a_i , then $M(i, j) = M(i-1, j)$.

Grading:

Recurrence 2 gets full 10 points. Recurrences 1 and 4 get 5 points since they have one of the two cases correct. Recurrence 3 gets 0 points.

4) 15 pts

This problem involves partitioning a given input string into disjoint substrings in the cheapest way. Let $x_1x_2 \dots x_n$ be a string, where particular value of x_i does not matter. Let $C(i,j)$ (for $i \leq j$) be the given precomputed cost of each substring $x_i \dots x_j$. A partition is a decomposition of a string into disjoint substrings. The cost of a partition is the sum of costs of substrings. The goal is to find the min-cost partition. An example. Let "ab" be a given string. This string can be partitioning in two different ways, such as, "a", "b", and "ab" with the following costs $C(1,1) + C(2,2)$ and $C(1,2)$ respectively.

a) Define (in plain English) subproblems to be solved. (5 pts)

Let subproblem $OPT(j)$ be the min-cost partition $x_1x_2 \dots x_j$

b) Write the recurrence relation for subproblems. (7 pts)

$$OPT(j) = \min_{1 \leq k \leq j} (OPT(k-1) + C(k, j))$$
$$OPT(0) = 0$$

c) Compute the runtime of the algorithm. (3 pts)

$$\Theta(n^2).$$

Common mistake:

If you write the recurrence based on two parameters of i, j for the corresponding subsequence $x_i \dots x_j$ like $opt(i, j)$ you might lose points from 4 to 7 points in part b, depending on your solution. In this case, you probably came up with complexity of $O(n^3)$ instead of $O(n^2)$ which is not optimum compared to the right solution and you might lose other points (2 to 3 points) in part c.

5)

You have two rooms to rent out. There are n customers interested in renting the rooms. The i^{th} customer wishes to rent one room (either room you have) for $d[i]$ days and is willing to pay $\text{bid}[i]$ for the entire stay. Customer requests are non-negotiable in that they would not be willing to rent for a shorter or longer duration. Devise a dynamic programming algorithm to determine the maximum profit that you can make from the customers over a period of D days.

Let $\text{OPT}(d1, d2, i)$ be the maximum profit obtainable with $d1$ remaining days for room 1 and $d2$ remaining days for room 2 using the first i customers.

a) Write the recurrence relation for subproblems. (7 pts)

$$\text{OPT}(d1, d2, i) = \max(\text{bid}[i] + \text{OPT}(d1 - d[i], d2, i - 1), \\ \text{bid}[i] + \text{OPT}(d1, d2 - d[i], i - 1), \\ \text{OPT}(d1, d2, i - 1))$$

Initial conditions

$$\text{OPT}(d1, d2, 0) = 0$$

$$\text{OPT}(d1, d2, i) = -\infty \text{ if } d1 < d[i] \text{ or } d2 < d[i]$$

b) Compute the runtime of the algorithm. (4 pts)

$$O(nD^2)$$

Grading Rubric:

Common mistakes:

1. Most common mistake is to model each room separately, so having two recurrence for two rooms. This is not true, and will lead to the same solution for each recurrence.
2. Another common mistake is to model the problem using 2D days (simply add up), this is also not true. i.e. If a customer wants to stay for 4 days and there are 2 days left in both rooms, then this approach of combining two rooms into one incorrectly suggests that the customer can stay.
3. Also, there are some students model the problem using three variables, (days, customers, rooms), this is not true.

4. Using one dimensional array to define subproblem is also wrong.

Credits:

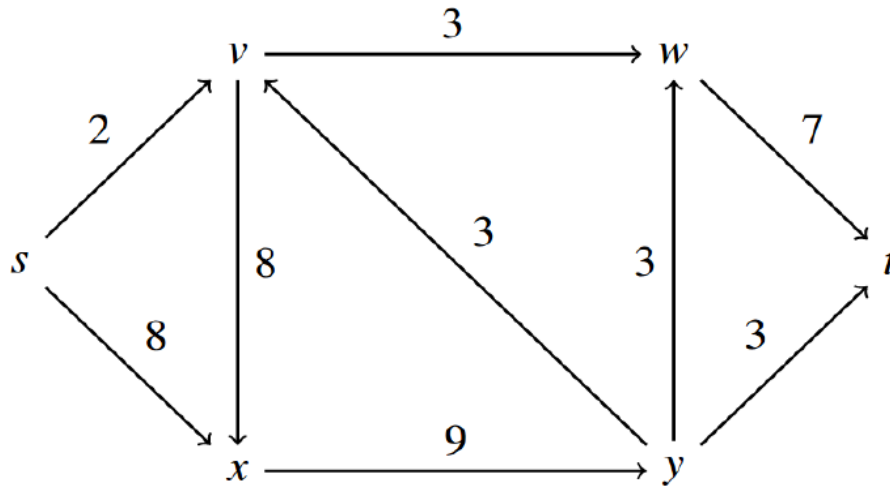
1. Without boundary condition, or incorrect boundary condition : -2 points
2. Every missing recurrence subproblem : -3 points. i.e. the correct recurrence should be $\max \{f_1, f_2, f_3\}$, but one of them is missing.
3. Incorrect time complexity: -4 points (no partial credit)

Partial credits:

For common mistakes mentioned above, give maximum of 6 points (depends on the correctness of recurrence equation and run time complexity, if the recurrence equation is not correct then no credit was given even though the complexity is correct)

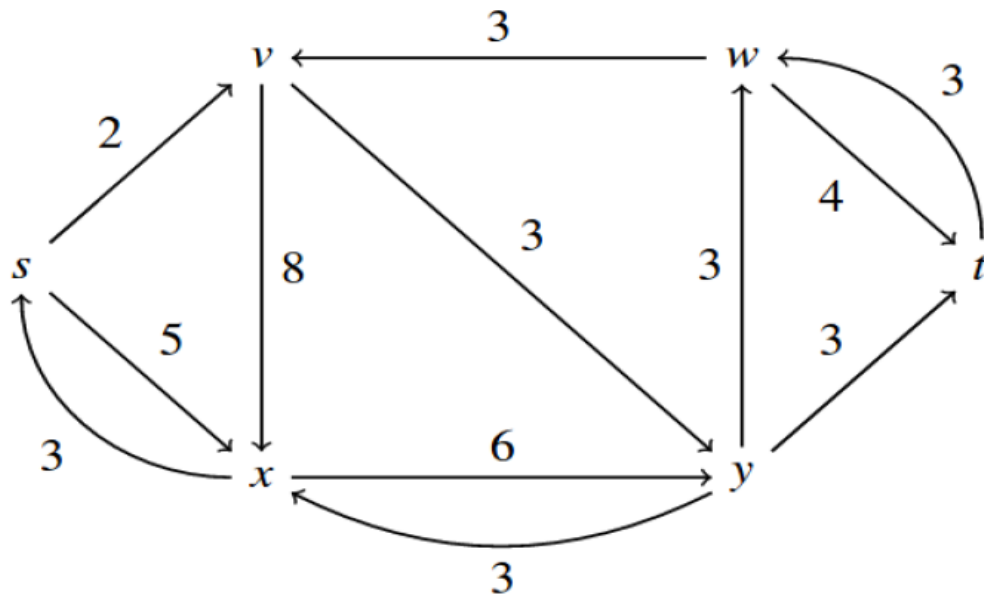
6) 15 pts

You are given the following graph. Each edge is labeled with the capacity of that edge.



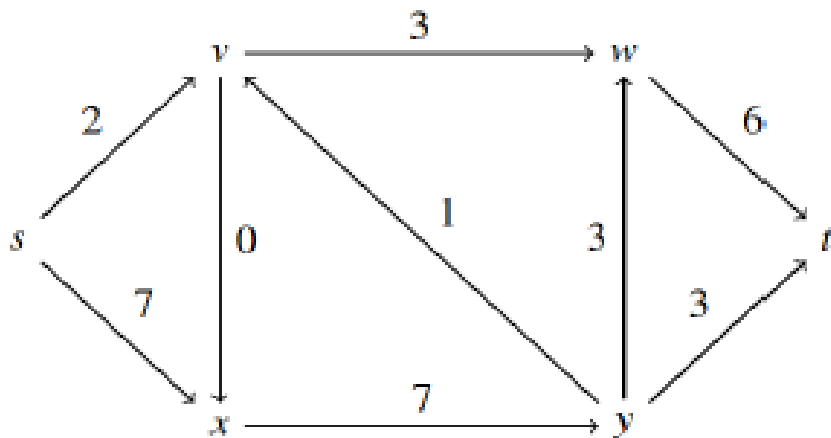
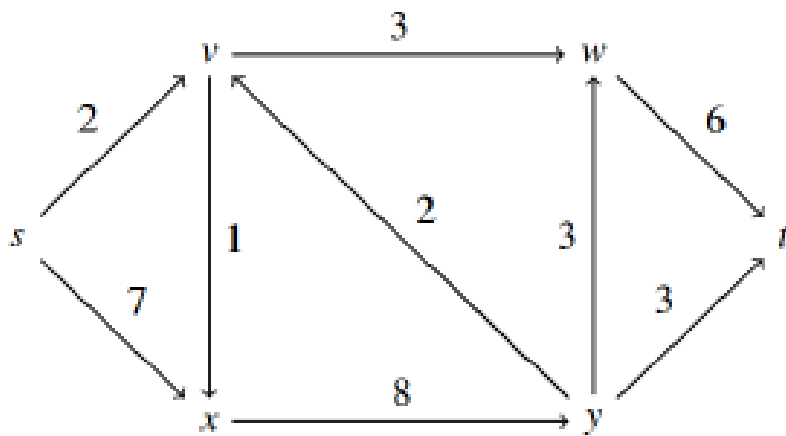
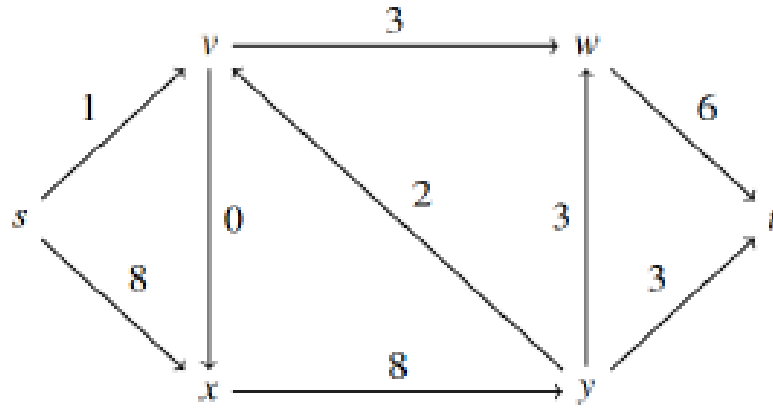
- a) Draw the corresponding residual graph after sending as much flow as possible along the path $s \rightarrow x \rightarrow y \rightarrow v \rightarrow w \rightarrow t$. (5 pts)

3

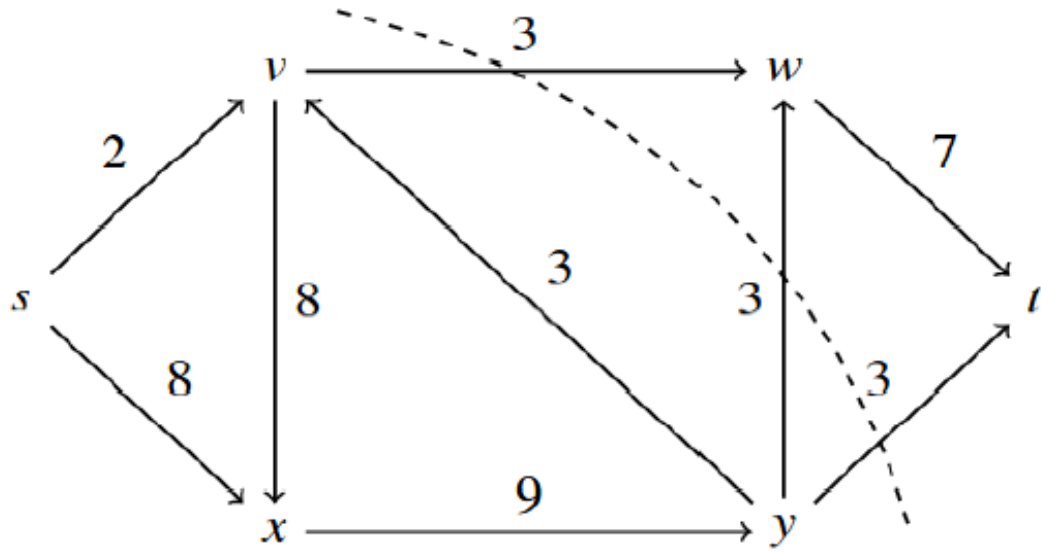


b) Find the value of a max-flow. (5 pts)

9, multiple answers



c) Find a min-cut? (5 pts)



7) 15 pts

Consider a drip irrigation system, which is an irrigation method that saves water and fertilizer by allowing water to drip slowly to the roots of plants. Suppose that the location of all drippers are given to us in terms of their coordinates (x^d, y^d) . Also, we are given locations of plants specified by their coordinates (x^p, y^p) .

A dripper can only provide water to plants within distance l . A single dripper can provide water to no more than n plants. However, we recently got some funding to upgrade our system with which we bought k monster drippers, which can provide water supply to three times the number of plants compared to standard drippers. So, we now have i standard drippers and k monster drippers.

Given the locations of the plants and drippers, as well as the parameters l and n , decide whether every plant can be watered simultaneously by a dripper, subject to the above mentioned constraints. Justify carefully that your algorithm is correct and can be obtained in polynomial time.

Solution:

Create a graph $G=(V, E)$

$V = \{\text{source } s, \text{ nodes } p_1, \dots, p_j \text{ for sets of plants, nodes } d_1, \dots, d_i \text{ for standard droppers, nodes for } d_{i+1}, \dots, d_{i+k}, \text{ and sink } t\}$ (1 pt)

$E = \{$

$c(s, p_x) = 1, \text{ for } x = 1 \text{ to } j$ (1 pt)

$c(p_x, d_y) = 1, \text{ for } x = 1 \text{ to } j, y = 1 \text{ to } i+k, (1 \text{ pt}) \text{ and } p_x, d_y \text{ are close enough (1 pt)}$

$c(d_y, t) = n, \text{ for } y = 1 \text{ to } i$ (2 pt)

$c(d_y, t) = 3n, \text{ for } y = i+1 \text{ to } i+k$ (2 pt)

$\}$

Use Ford-Fulkerson algorithm to find maximum integer flow f in G and justification. (7 pt)#

CS570 Spring 2019: Analysis of Algorithms Exam II

	Points		Points
Problem 1	20	Problem 5	20
Problem 2	6	Problem 6	14
Problem 3	20		
Problem 4	20		
	Total	100	

Instructions:

1. This is a 2-hr exam. Closed book and notes
2. If a description to an algorithm or a proof is required please limit your description or proof to within 150 words, preferably not exceeding the space allotted for that question.
3. No space other than the pages in the exam booklet will be scanned for grading.
4. If you require an additional page for a question, you can use the extra page provided within this booklet. However please indicate clearly that you are continuing the solution on the additional page.
5. Do not detach any sheets from the booklet. Detached sheets will not be scanned.
6. If using a pencil to write the answers, make sure you apply enough pressure so your answers are readable in the scanned copy of your exam.
7. Do not write your answers in cursive scripts.

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[**TRUE/FALSE**]

The Ford-Fulkerson algorithm can be used to find the maximum flow through a graph with cycles.

[**TRUE/FALSE**]

In a flow network where all edge capacities are unique, the min cut will be unique.

[**TRUE/FALSE**]

A Dynamic Programming algorithm with n^3 unique sub-problems will run in $O(n^4)$ time.

[**TRUE/FALSE**]

It is possible for a dynamic programming algorithm to have exponential running time.

[**TRUE/FALSE**]

Let G be a weighted directed graph with exactly one source s and exactly one sink t . Let (A,B) be a maximum cut in G , that is, A and B are disjoint sets whose union is V , $s \in A$, $t \in B$, and the sum of the weights of all edges from A to B is the maximum for any two such sets. Now let H be the weighted directed graph obtained by adding 1 to the weight of each edge in G . Then (A,B) must still be a maximum cut in H .

[**TRUE/FALSE**]

The recurrence $OPT(j) = \max_{1 \leq i \leq j} \{A[i] - B[j - i] + OPT(j - i)\}$ where A and B are fixed Input arrays, will lead to an $O(n)$ dynamic programming algorithm.

[**TRUE/FALSE**]

In a graph with more than one minimum cut, if we take any edge e that is part of the (A,B) minimum cut and increase the capacity of that edge by any integer $k \geq 1$, then (A,B) will no longer be a minimum cut.

[**TRUE/FALSE**]

The Ford-Fulkerson algorithm (without scaling) can be used to efficiently solve the maximum matching problem in a bipartite graph.

[**TRUE/FALSE**]

The space efficient version of the sequence alignment algorithm (which uses a combination of divide and conquer and dynamic programming) has the same running time complexity as the basic dynamic programming algorithm for this problem.

[**TRUE/FALSE**]

Suppose, increasing the capacity of edge e by 1, increases value of max flow f by 1. Then it must be that $f(e) = \text{capacity}(e)$.

2) 6 pts

Let $G = (V, E)$ be a flow network with source s , sink t , and integer capacities.

Suppose that we are given a maximum flow f in G . Now, we increase the capacity of a single edge e by 1.

a) If $f(e) < \text{Capacity}(e)$, can we say that f will still be a maximum flow in G ? (2pts)

Yes.

Remarks: this part is not graded since there was confusion about whether the capacity was before the flow or after. So “No” is also accepted as correct answer.

b) If $f(e) = \text{Capacity}(e)$, can we say that f will no longer be a maximum flow in G ? (2 pts)

No.

c) Give an $O(V + E)$ -time algorithm to find a new maximum flow in G . (2 pts)

Find new G_f and see if there is a path from s to t . If there is, do one more step of augmentation.

Rubric: All-or-nothing. No partial credit.

3) 20 pts

We are given a string $S = s_1s_2s_3\dots s_n$, and we want to insert some characters such that the resulting string is a palindrome. Give an efficient dynamic programming algorithm to determine the minimum number of insertions.

Definition: A palindrome is a string that reads the same forward and backward such as ACCBCCA.

Example: Input: ACB Output: 2 (ACBCA)

a) Define (in plain English) subproblems to be solved. (5 pts)

$\text{opt}(i,j)$ = minimum number of insertions to make substring $s[i\dots j]$ a palindrome.

grading rubric:

3pts if using $\text{opt}(i)$ to represent the number of insertions to make substring $s[0\dots i]$ a palindrome

reverse the string is ok and is graded in a similar scheme.

0pts everything else.

b) Write the recurrence relation for subproblems. (7 pts)

if $s_i == s_j$: $\text{opt}(i,j) = \text{opt}(i+1, j-1)$
else: $\text{opt}(i,j) = \min(\text{opt}(i+1,j), \text{opt}(i, j-1)) + 1$

grading rubric:

$s_i == s_j$: 2pts

$\text{opt}(i+1, j)$: 2pts

$\text{opt}(i, j-1)$: 2pts

+1: 7pts

reverse the string and compare are ok and is graded in a similar scheme. use sigma (or other constants) to represent mismatch penalty is ok.

c) Using the recurrence formula in part b, write pseudocode to compute the minimum number of insertions. (5 pts)

Make sure you have initial values properly assigned. (3 pts)

for $i=1$ to n :

 for $j=1$ to n :

$\text{opt}(i,j) = 0$

```

for j=1 to n:
    for i = j-1 to 1:
        if si == sj:
            if i < j-1:
                opt(i,j) =opt(i+1, j-1)
            else:
                opt(i,j) = 0
        else:
            opt(i,j) = min(opt(i+1,j), opt(i,j-1))+1
return opt(1,n)

```

grading rubrics:

initialization: 3pts (need to initialize to 0).

recursion: if si == sj: opt(i,j) =opt(i+1, j-1) 2pts; opt(i,j) = min(opt(i+1,j), opt(i,j-1))+1 3pts.

4) 20 pts

The Laver cup is a tennis tournament that takes place every year between Team Europe and Team US, each consisting of k players. Each player participating has a world ranking. A match between players of ranking r_i and r_j is fair if $|r_i - r_j| \leq d$. The tournament must consist of n matches, each between a player from Team EU and a player from Team US. Describe an efficient algorithm that, given the rankings of all the players, determines if it is possible to schedule n fair matches between the two teams such that each player plays at least one match but no more than m matches, and no two players play each other more than once.

A4.

- bipartite graph with k players of team US on left and k players of team EU on right.
- connect with an edge each US player (us_i) to the corresponding EU player (eu_j) whose skill level is within d . Each such edge has weight one to ensure at most one match between any two players in opposing teams.
- two nodes source and sink. Where edges (s, us_i) have a lower bound of 1 and capacity m . Likewise edges (eu_j, t) have lower bound 1 and capacity m .
- Add a demand of n at t and supply of n at s .

To solve: Convert to the problem of circulation with demand.

Claim: a matching exists if there exists a feasible circulation in the network.

Grading Rubric:

- S->US->EU->T (or S->EU->US->T) structure

2. Demand of $-n$ for S and n for T
3. Connecting players with the restriction of fair matching
4. For $S \rightarrow US$ and $EU \rightarrow T$ edges, capacity= m and lower bound= 1 ; for $US \rightarrow EU$ edges, capacity= 1 and no lower bound
5. Final claim indicating a maxflow (available flow) of size n . Note that $\text{maxflow} \geq n$ is incorrect, because when $\text{maxflow} > n$, a flow of n is not necessarily possible in the network (e.g. when $k > n$, we get $\text{maxflow} \geq k > n$, but a flow of size n is impossible).

4 points each for each correct use of above. No partial credit!

Algorithms other than network flow are considered incorrect/inefficient, with 0 point given.

20 pts

- 5) Your company, Stuckbars Coffee and Toffee is planning to open several stores on Main Street. Main Street has n blocks $1, 2, \dots, n$ from East to West, and for each block i you know the revenue $r_i > 0$ you expect from a store on this block. But if you open a store on block i , you cannot open another store in block i , or the two blocks to its East or the two blocks to its West. You have designed a dynamic programming algorithm for finding the store locations that maximizes total revenue, but you spilled coffee on it and now you have to reconstruct it. Here are the steps you need to follow.

All solution more complex than the linear solution are considered incorrect. The incorrect part will lead to score reduction in every section. If the solution uses other methods, a student must show it has a complexity of $O(n)$. Otherwise, it is very likely to get a lower score if request to be regraded.

- d) Define (in plain English) subproblems to be solved. (4 pts)

Choice of subproblem: $R[i]$ = the maximum total revenue if stores can only be opened on blocks $1, \dots, i$

Rubric:

If state $\text{opt}(i)$ means max revenue for store 1 to i , or i to n . (full credits)

If state placement on a store on i th block but not mention 1 to i or i to n . (-1 or -2)

Other complicated sub problem or ambiguous statement (-2 or -3)

If not a subproblem: -4

- e) Write the recurrence relation for subproblems. (6 pts)

Recurrence relation: $R[i] = \max \{ R[i - 1], r_i + R[i - 3] \}$

Rubric:

If $\max(r_i + \text{opt}(i-3), \text{opt}(i-1))$ (full credits)

Write $\max(\text{opt}(i-1), \text{opt}(i-2), \text{opt}(i-3) + r_i)$ is ok, since it doesn't increase complexity

If writes $i-3$ as $i-2$: (-1 or -2)

Other incorrect answer (-3 to -5)

Empty (-6)

- f) Using the recurrence formula in part b, write pseudocode to compute the maximum revenue possible. (4 pts)

Make sure you have initial values properly assigned. (2 pts)

$R[1] = r_1$ $R[2] = \max(r_1, r_2)$ $R[3] = \max(r_1, r_2, r_3)$

For $i = 4, \dots, n$;

$R[i] = \max \{ R[i - 1], r_i + R[i - 3] \}$

Initializing n, n-1 and n-2, or R[0], R[1], R[2], or R[<0],R[1] are all correct

If index is incorrect (-1 or -2)

If other incorrect methods which have higher complexity (-3 or -4)

Initialization as R[3] = r3 R[2]=r2 (-1 or -2)

- g) Using the results in part c, write pseudocode to determine the store locations.
(2 pts)

$$\text{Recover solution: } prev[i] = \begin{cases} i - 3, & \text{if } R[i] = r_i + R[i - 3] \\ i - 1, & \text{otherwise} \end{cases}$$

Rubric:

Only mention Trace back (-2 or -1)

Wrong in bound (-1)

Not using result in c or empty (-2)

Include i-1 or i-3 in the location(-1 or -2) consider a case: r1=1, r2=1, r3=1000,

r4=1, r5=1, r6=1, r7=1, r8=1000, r9=1, r10=1

we should only select r3 and r8.

- h) Compute the runtime of the algorithm described in part c and state whether your solution runs in polynomial time or not (2 pts)

Running time: O(n), since each subproblem takes constant time to compute based on smaller subproblems, and there are n subproblems to compute.

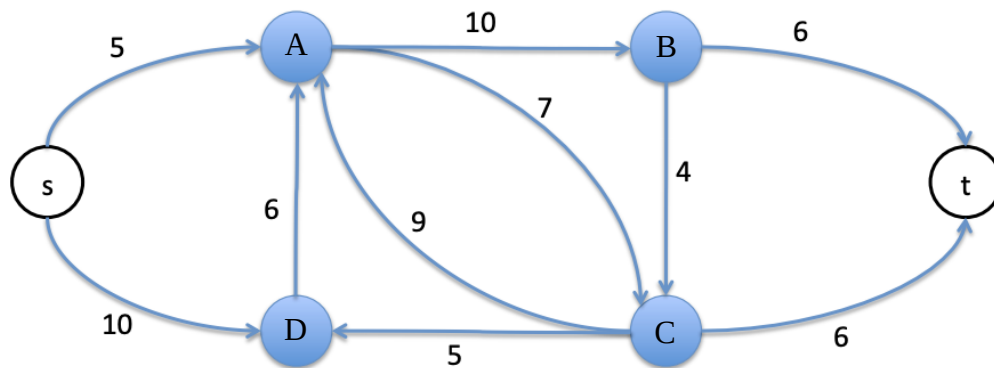
Rubric:

If not o(n) (-1)

If not strong polynomial (-1)

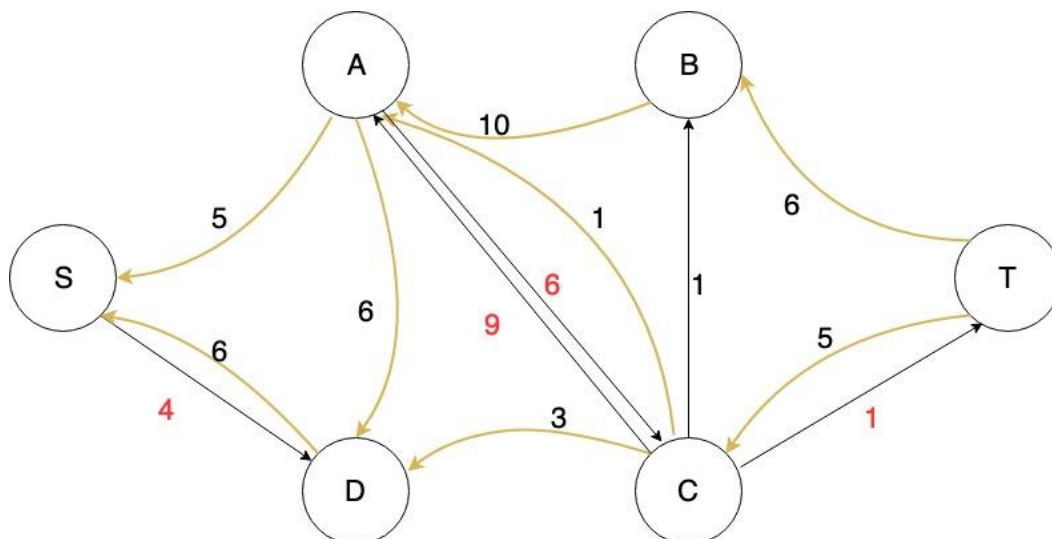
10 pts

6) For the following flow network, with edge capacities as shown,

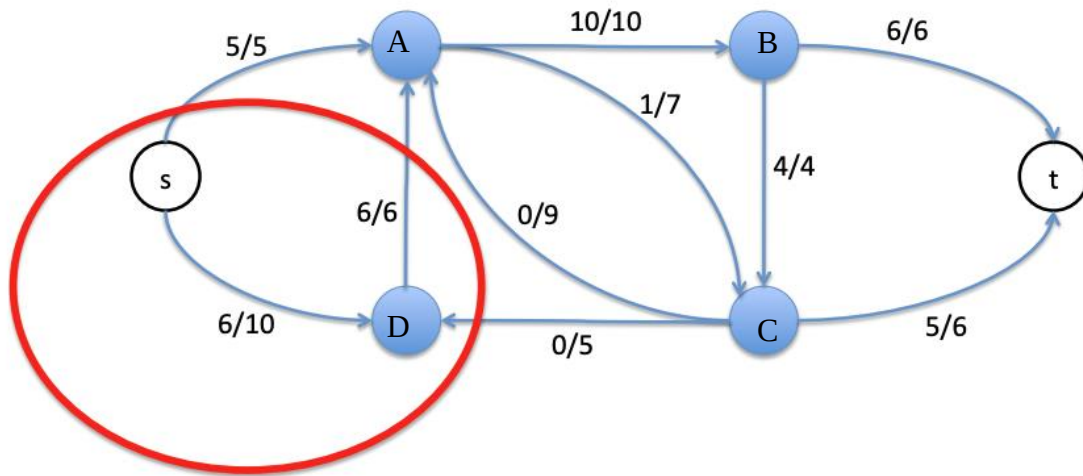


a) Find a minimum s - t cut in the network and indicate what its capacity is.
(Clearly show which nodes are in which set)

Solution



This is the final residual graph, yellow ones are back edge, red values are residual capacity.



The cut has cost 11, and we can show that it is the minimum cut by demonstrating a flow with value 11—see above for the flow assignment.

Two sets: $\text{set1}\{D\}, \text{set2}\{A, C, B\}$

Since the maximum-flow is equal to the minimum cut, this proves that 11 is the minimum cut.

b) Describe how you would determine if this is the only minimum cut in G .

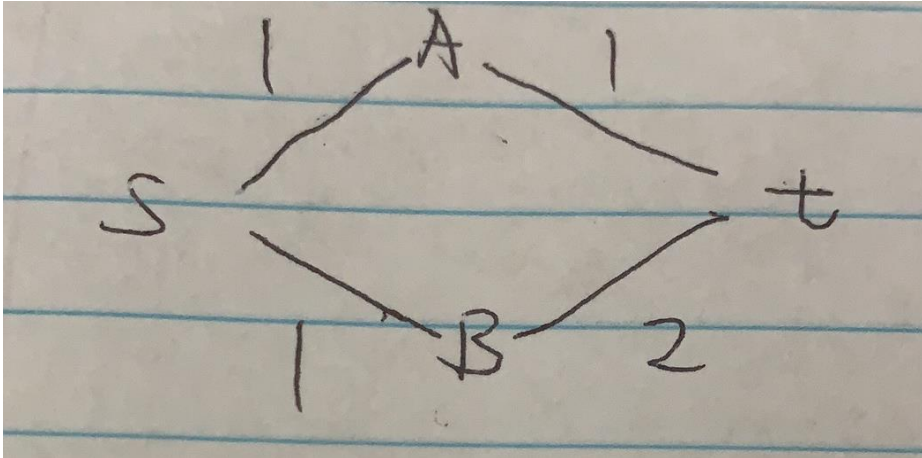
Solution1. BFS from source to output the min-cut. Then BFS from sink to output another min-cut. If same then this is the only mincut, else multiple min-cuts exist.

Solution2. For every edge e in $(\text{set1}, \text{set2})$, increase the capacity of that edge and compute the value of the max-flow again. If the value does not increase, then it means that there is another min-cut that there was another min-cut in the original graph that did not include this edge, and hence the min-cut was not unique. If for all the edges in $(\text{set1}, \text{set2})$ the value of the max-flow increases, then the min-cut is unique.

Common mistakes:

1. Increase value on only one of the edges in min-cut. Or increase all of them at the same time instead of one by one.

Counterexample:



2. Try to find augment path in other orders to find different min-cut.
Counterexample: $s-1-A-2-B-1-t$

Rubric:

a.

Steps for 5 points.

Min-cut for 3 points.

Capacity for 2 points.

b.

Partial score is given only on common mistake 1.

CS570
Analysis of Algorithms
Fall 2014
Exam II

Name: _____

Student ID: _____

Email: _____

Wednesday Evening Section

	Maximum	Received
Problem 1	20	
Problem 2	16	
Problem 3	16	
Problem 4	20	
Problem 5	16	
Problem 6	12	
Total	100	

Instructions:

1. This is a 2-hr exam. Closed book and notes
2. If a description to an algorithm or a proof is required please limit your description or proof to within 150 words, preferably not exceeding the space allotted for that question.
3. No space other than the pages in the exam booklet will be scanned for grading.

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[FALSE]

If an iteration of the Ford-Fulkerson algorithm on a network places flow 1 through an edge (u, v) , then in every later iteration, the flow through (u, v) is at least 1.

[TRUE/]

For the recursion $T(n) = 4T(n/3) + n$, the size of each subproblem at depth k of the recursion tree is $n/3^{k-1}$.

[FALSE]

For any flow network G and any maximum flow on G , there is always an edge e such that increasing the capacity of e increases the maximum flow of the network.

[FALSE]

The asymptotic bound for the recurrence $T(n) = 3T(n/9) + n$ is given by $\Theta(n^{1/2} \log n)$.

[FALSE]

Any Dynamic Programming algorithm with n subproblems will run in $O(n)$ time.

[FALSE]

A pseudo-polynomial time algorithm is always slower than a polynomial time algorithm.

[TRUE/]

The sequence alignment algorithm can be used to find the longest common subsequence between two given sequences.

[FALSE]

If a dynamic programming solution is set up correctly, i.e. the recurrence equation is correct and each unique sub-problem is solved only once (memoization), then the resulting algorithm will always find the optimal solution in polynomial time.

[TRUE/]

For a divide and conquer algorithm, it is possible that the divide step takes longer to do than the combine step.

[TRUE/]

Maximum value of an $s - t$ flow could be less than the capacity of a given $s - t$ cut in a flow network.

2) 16 pts

Recall the Bellman-Ford algorithm described in class where we computed the shortest distance from all points in the graph to t . And recall that we were able to find all shortest distance to t with only $O(n)$ memory.

How would you extend the algorithm to compute both the shortest distance and to find the actual shortest paths from all points to t with only $O(n)$ memory?

We need an array of size n to hold a pointer to the neighbor that gives us the shortest distance to t . Initially all pointers are set to Null. Whenever a node's distance to t is reduced, we update the pointer for that node and point it to the node that is giving us a lower distance to t . Once all shortest distances are computed, to find a path from any node v to t , one can simply follow these pointers to reach t on the shortest path.

3) 16 pts

During their studies, 7 friends (Alice, Bob, Carl, Dan, Emily, Frank, and Geoffrey) live together in a house. They agree that each of them has to cook dinner on exactly one day of the week. However, assigning the days turns out to be a bit tricky because each of the 7 students is unavailable on some of the days. Specifically, they are unavailable on the following days (1 = Monday, 2 = Tuesday, ..., 7 = Sunday):

- Alice: 2, 3, 4, 5
- Bob: 1, 2, 4, 7
- Carl: 3, 4, 6, 7
- Dan: 1, 2, 3, 5, 6
- Emily: 1, 3, 4, 5, 7
- Frank: 1, 2, 3, 5, 6
- Geoffrey: 1, 2, 5, 6

Transform the above problem into a maximum flow problem and draw the resulting flow network. If a solution exists, the flow network should indicate who will cook on each day; otherwise it must show that a feasible solution does not exist

Solution:

I will use the initials of each person's name to refer to them in this solution.

Construct a graph $G = (V, E)$. V consists of 1 node for each person (let us denote this set by $P = \{A, B, C, D, E, F, G\}$), 1 node for each day of the week (let's call this set $D = \{1, 2, 3, 4, 5, 6, 7\}$), a source node s , and a sink node t . Connect s to each node p in P by a directed (s, p) edge of unit capacity. Similarly, connect each node d in D to t by a directed (d, t) edge of unit capacity. Connect each node p in P by a directed edge of unit capacity to those nodes in D when p is **available** to cook. This completes our construction of the flow network. I am omitting the actual drawing of G here.

Finding a max-flow of value 7 in G translates to finding a feasible solution to the allocation of cooking days problem. Since there can be at most unit flow coming into any node p in P , a maximum of unit flow can leave it. Similarly, at most a flow of value 1 can flow into any node d in D because a maximum of unit flow can leave it. Thus, a max-flow of value 7 means that there exists a flow-carrying s - p - d - t path for each p and d . Any (p, d) edge with unit flow indicates that person p will cook on day d .

The following lists one possible max-flow of value 7 in G :

Send unit flow on each (s, p) edge and each (d, t) edge. Also send unit flow on the following (p, d) edges: (A, 6), (B, 5), (C, 1), (D, 4), (E, 2), (F, 7), (G, 3)

4) 20 pts

Suppose that there are n asteroids that are headed for earth. Asteroid i will hit the earth in time t_i and cause damage d_i unless it is shattered before hitting the earth, by a laser beam of energy e_i . Engineers at NASA have designed a powerful laser weapon for this purpose. However, the laser weapon needs to charge for a duration ce before firing a beam of energy e . Can you design a dynamic programming based pseudo-polynomial time algorithm to decide on a firing schedule for the laser beam to minimize the damage to earth? Assume that the laser is initially uncharged and the quantities c, t_i, d_i, e_i are all positive integers. Analyze the running time of your algorithm. You should include a brief description/derivation of your recurrence relation. Description of recurrence relation = 8pts, Algorithm = 6pts, Run Time = 6pts

Solution 1 (Assuming that the laser retains energy between firing beams): Sort all asteroids by the time t_i . Label the asteroids from 1 to n and without loss of generality assume $t_1 < t_2 < \dots < t_n$. Also assume that if asteroid i is destroyed then it is done exactly at time t_i (if the laser continuously accumulates energy then the destruction order of the asteroids does not change even if i^{th} asteroid is shot down before time t_i).

Define $OPT(i, T)$ as the minimum possible damage caused to earth due to asteroids $i, i + 1, \dots, n$ if $\frac{T}{c}$ energy is left in the laser just before time t_i . We want the solution corresponding to $OPT(1, t_1)$. If $T \geq ce_i$ and the i^{th} asteroid is destroyed then $OPT(i, T) = OPT(i + 1, T - ce_i + t_{i+1} - t_i)$, otherwise $OPT(i, T) = d_i + OPT(i + 1, T + t_{i+1} - t_i)$. Hence,

$$OPT(i, T) = \begin{cases} d_i + OPT(i + 1, T + t_{i+1} - t_i), & T < ce_i \\ \min\{d_i + OPT(i + 1, T + t_{i+1} - t_i), OPT(i + 1, T - ce_i + t_{i+1} - t_i)\}, & T \geq ce_i \end{cases}$$

Boundary condition: $OPT(n, T) = 0$ if $T \geq ce_n$ and $OPT(n, T) = d_n$ if $T < ce_n$, since if there is enough energy left to destroy the last asteroid then it is always beneficial to do so. Furthermore, $T \leq t_n$ since a maximum of $\frac{t_n}{c}$ energy is accumulated by the laser before the last asteroid hits the earth and $T \geq 0$ since the left over energy in the laser is always non-negative.

Algorithm:

- i. Initialize $OPT(n, T)$ according to the boundary condition above for $0 \leq T \leq t_n$.
- ii. For each $n - 1 \geq i \geq 1$ and $0 \leq T \leq t_n$ populate $OPT(i, T)$ according to the recurrence defined above.
- iii. Trace forward through the two dimensional OPT array starting at $(1, t_1)$ to determine the firing sequence. For $1 \leq i \leq n - 1$, destroy the i^{th} asteroid with $\frac{T}{c}$ energy left if and only if $OPT(i, T) = OPT(i + 1, T - ce_i + t_{i+1} - t_i)$. Destroy the n^{th} asteroid only if $T \geq ce_n$.

Complexity: Step (i) initializes t_n values each taking constant time. Step (ii) computes $(n - 1)t_n$ values, each taking one invocation of the recurrence and hence is done in $O(nt_n)$ time. Trace back takes $O(n)$ time since the decision for each i takes constant time. Initial sorting takes $O(n \log n)$ time. Thus overall complexity is $O(nt_n + n \log n)$.

Solution 2 (Assuming that the laser does not retain energy left over after firing and if asteroid i is destroyed then it is done exactly at time t_i): Sort all asteroids by the time t_i . Label the asteroids from 1 to n and without loss of generality assume $t_1 < t_2 < \dots < t_n$. In contrast to Solution 1, the destroying sequence will change if we are free to destroy asteroid i before time t_i (this case is not solved here).

Define $OPT(i)$ to be the minimum possible damage caused to earth due to the first i asteroids. We want the solution corresponding to $OPT(n)$. If the i^{th} asteroid is not destroyed either by choice or because $t_i < ce_i$ then $OPT(i) = d_i + OPT(i - 1)$. On the other hand, if the i^{th} asteroid is destroyed ($t_i \geq ce_i$ is necessary) then none of the asteroids arriving between times $t_i - ce_i$ and t_i (both exclusive) can be destroyed. Letting $p[i]$ denote the largest positive integer such that $t_{p[i]} \leq t_i - ce_i$ (and $p[i] = 0$ if no such integer exists), we have $OPT(i) = OPT(p[i]) + \sum_{j=p[i]+1}^{i-1} d_j$ if $p[i] \leq i - 2$ and $OPT(i) = OPT(i - 1)$ if $p[i] = i - 1$. Hence for $i \geq 1$,

$$OPT(i) = \begin{cases} OPT(i - 1), & t_i \geq ce_i \text{ and } p[i] = i - 1 \\ d_i + OPT(i - 1), & t_i < ce_i \\ \min \left\{ d_i + OPT(i - 1), OPT(p[i]) + \sum_{j=p[i]+1}^{i-1} d_j \right\}, & t_i \geq ce_i \text{ and } p[i] \leq i - 2 \end{cases}$$

Boundary condition: $OPT(0) = 0$ since no asteroids means no damage.

Algorithm:

- i. Form the array p element-wise. This is done as follows.
 - a. Set $p[1] = 0$.
 - b. For $2 \leq i \leq n$, binary search for $t_i - ce_i$ in the sorted array $t_1 < t_2 < \dots < t_n$. If $t_i - ce_i < t_1$ then set $p[i] = 0$ else record $p[i]$ as the index such that $t_{p[i]} \leq t_i - ce_i < t_{p[i]+1}$.
- ii. Form array D to store cumulative sum of damages. Set $D[0] = 0$ and $D[j] = D[j - 1] + d_j$ for $1 \leq j \leq n$.
- iii. Set $OPT(0) = 0$ and for $1 \leq i \leq n$ populate $OPT(i)$ according to the recurrence defined above, computing $\sum_{j=p[i]+1}^{i-1} d_j$ as $D[i - 1] - D[p[i]]$.
- iv. Trace back through the one dimensional OPT array starting at $OPT(n)$ to determine the firing sequence. Destroy the first asteroid if and only if $OPT(1) = 0$. For $2 \leq i \leq n$, destroy the i^{th} asteroid if and only if either $OPT(i) = OPT(i - 1)$ with $p[i] = i - 1$ or $OPT(i) = OPT(p[i]) + D[i - 1] - D[p[i]]$ with $p[i] \leq i - 2$.

Complexity: initial sorting takes $O(n \log n)$. Construction of array p takes $O(\log n)$ time for each index and hence a total of $O(n \log n)$ time. Forming array D is done in $O(n)$ time. Using array D , $OPT(i)$ can be populated in constant time for each $1 \leq i \leq n$ and hence step (iii) takes $O(n)$ time. Trace back takes $O(n)$ time since the decision for each i takes constant time. Therefore, overall complexity is $O(n \log n)$.

5) 16 pts

Consider a two-dimensional array $A[1:n, 1:n]$ of integers. In the array each row is sorted in ascending order and each column is also sorted in ascending order. Our goal is to determine if a given value x exists in the array.

- a. One way to do this is to call binary search on each row (alternately, on each column). What is the running time of this approach? [2 pts]
- b. Design another divide-and-conquer algorithm to solve this problem, and state the runtime of your algorithm. Your algorithm should take strictly less than $O(n^2)$ time to run, and should make use of the fact that each row and each column is in sorted order (i.e., don't just call binary search on each row or column). State the run-time complexity of your solution.

a) $O(n \log n)$.

b) Look at the middle element of the full matrix. Based on this, you can either eliminate $A[1.. \frac{n}{2}, 1.. \frac{n}{2}]$ or $A[\frac{n}{2}..n, \frac{n}{2}..n]$. If x is less than middle element then you can eliminate $A[1.. \frac{n}{2}, 1.. \frac{n}{2}]$. If x is greater middle element then you can eliminate $A[\frac{n}{2}..n, \frac{n}{2}..n]$. You can then recursively search in the remaining three $\frac{n}{2} \times \frac{n}{2}$ matrices. The total runtime is $T(n) = 3T(\frac{n}{2}) + O(1)$, $T(n) = O(n^{\log_2 3})$.

6) 12 pts

Consider a divide-and-conquer algorithm that splits the problem of size n into 4 sub-problems of size $n/2$. Assume that the divide step takes $O(n^2)$ to run and the combine step takes $O(n^2 \log n)$ to run on problem of size n . Use any method that you know of to come up with an upper bound (as tight as possible) on the cost of this algorithm.

Solution: Use the generalized case 2 of Master's Theorem. For $T(n) = aT\left(\frac{n}{b}\right) + \Theta(n^{\log_b a} \log^k n)$ with $k \geq 0$, we have $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.

The divide and combine steps together take $O(n^2 \log n)$ time and the worst case is that they actually take $\Theta(n^2 \log n)$ time. Hence the recurrence for the given algorithm is $T(n) = 4T\left(\frac{n}{2}\right) + \Theta(n^2 \log n)$ in the worst case. Comparing with the generalized case, $a = 4, b = 2, k = 1$ and so $T(n) = \Theta(n^2 \log^2 n)$. Since this expression for $T(n)$ is the worst case running time, an upper bound on the running time is $O(n^2 \log^2 n)$.

Additional Space

CS570
Analysis of Algorithms
Spring 2015
Exam II

Name: _____

Student ID: _____

Email Address: _____

_____ **Check if DEN Student**

	Maximum	Received
Problem 1	20	
Problem 2	20	
Problem 3	20	
Problem 4	20	
Problem 5	20	
Total	100	

Instructions:

1. This is a 2-hr exam. Closed book and notes
2. If a description to an algorithm or a proof is required please limit your description or proof to within 150 words, preferably not exceeding the space allotted for that question.
3. No space other than the pages in the exam booklet will be scanned for grading.
4. If you require an additional page for a question, you can use the extra page provided within this booklet. However please indicate clearly that you are continuing the solution on the additional page.

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[FALSE]

A flow network with unique edge capacities has a unique min cut.

[TRUE/]

If a problem can be solved by dynamic programming, then it can always be solved by exhaustive search (Brute Force).

[TRUE/]

A divide and conquer algorithm acting on an input size of n can have a lower bound less than $\Theta(n \log n)$.

[FALSE]

If a flow in a network has a cycle, this flow is not a valid flow.

[FALSE]

In the divide and conquer algorithm to compute the closest pair among a given set of points on the plane, if the sorted order of the points on both X and Y axis are given as an added input, then the running time of the algorithm improves to $O(n)$.

[TRUE/]

In a flow network, an edge that goes straight from s to t is always saturated when maximum $s - t$ flow is reached.

[FALSE]

The Bellman-Ford algorithm always fails to find the shortest path between two nodes in a graph if there is a negative cycle present in the graph.

[TRUE/]

If f is a max $s-t$ flow of a flow network G with source s and sink t , then the capacity of the min $s-t$ cut in the residual graph G_f is 0.

[FALSE]

In a dynamic programming solution, the space requirement is always at least as big as the number of unique sub problems.

[FALSE]

Decreasing the capacity of an edge that belongs to a min cut in a flow network may not result in decreasing the maximum flow.

2) 20 pts

A city is located at one node x in an undirected network $G = (V, E)$ of channels. There is a big river beside the network. In the rainy season, the flood from the river flows into the network through a set of nodes Y . Assume that the flood can only flow along the edges of the network. Let c_{uv} (integer value) represent the minimum effort (counted in certain effort unit) of building a dam to stop the flood flowing through edge (u, v) . The goal is to determine the minimum total effort of building dams to prevent the flood from reaching the city. Give a pseudo-polynomial time algorithm to solve this problem. Justify your algorithm.

Algorithm:

1. Add node s , and add edges from s to each node in Y .
2. Set the capacity of each of these new edges as infinity.
3. Set city node x as the sink node. Then the flood network G becomes a larger network G' with source s and sink x .
4. Find the min s - x cut on G' by running a polynomial time max-flow algorithm, where you can treat the cost of building a dam on each edge to be the capacity of this edge.
5. Build a dam on each edge in the min-cut's cut-set.

Complexity:

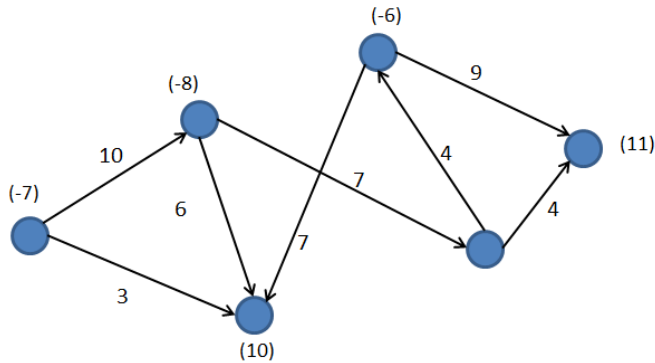
Since the construction of the new edges takes $O(|Y|)$ times, together with a polynomial time max-flow algorithm to find the min-cut, the entire algorithm takes polynomial time.

Justification:

We're simply trying to separate x from Y by choosing edges with the minimum cost, which is a min-cut problem. But min-cut only works when Y is a single node. We fix this by creating a super source s directly connected to Y . But we want to make sure the min-cut's cut-set doesn't include any edge connecting s , because these edges do not belong to G . This is why we put the capacities arbitrarily large on these edges. Min-cut would then find the min-cost way to separate x from Y .

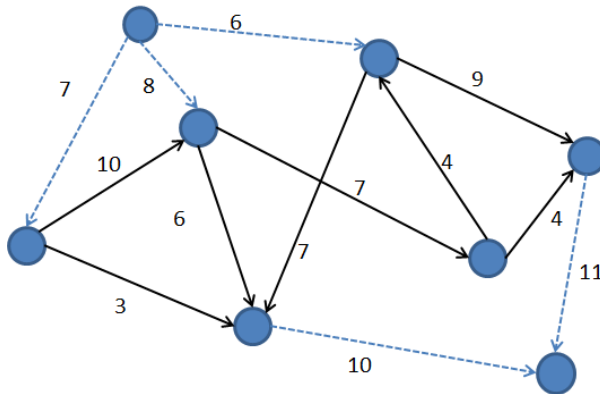
3) 20 pts

The following graph G is an instance of a circulation problem with demands. The edge weights represent capacities and the node weights (in parentheses) represent demands. A negative demand implies source.



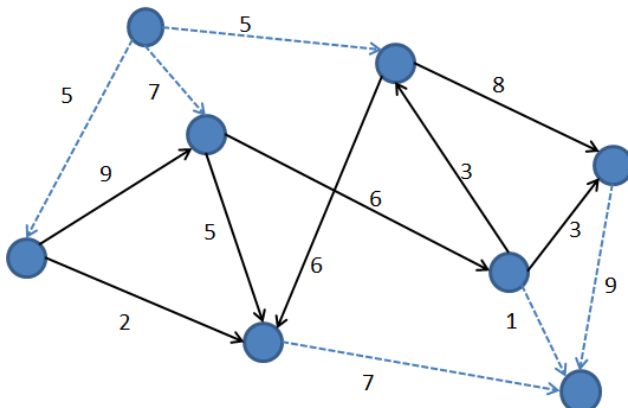
(i) Transform this graph into an instance of max-flow problem.

Solution:



(ii) Now, assume that each edge of G has a constraint of lower bound of 1 unit, i.e., one unit must flow along all edges. Find the new instance of max-flow problem that includes the lower bound constraint.

Solution:



4) 20 pts

There is a series of activities lined up one after the other, $J_1, J_2, \dots, J_n$. The i^{th} activity takes T_i units of time, and you are given M_i amount of money for it. Also for the i^{th} activity, you are given N_i , which is the number of immediately following activities that you cannot take if you perform that i^{th} activity. Give a dynamic programming solution to maximize the amount of money one can make in T units of time. Note that an activity has to be completed in order to make any money on it. State the runtime of your algorithm.

Solution:

Recurrence formula:

If $T_i > t$ then $\text{opt}(i, t) = \text{opt}(i+1, t)$,

Otherwise, $\text{opt}(i, t) = \max(\text{opt}(i+1, t), \text{opt}(i + N_i + 1, t - T_i) + M_i)$

Boundary conditions: $\text{opt}(i, t) = 0$ for $i > n$ and all t ,

$\text{opt}(i, t) = 0$ for $t < 1$ and all i

To find the value of the optimal solution:

for $i = n$ to 1 by -1

for $t = 1$ to T

 If $T_i > t$ then $\text{opt}(i, t) = \text{opt}(i+1, t)$,

 Otherwise, $\text{opt}(i, t) = \max(\text{opt}(i+1, t), \text{opt}(i + N_i + 1, t - T_i) + M_i)$

end for

end for

$\text{opt}(1, T)$ will hold the value of the optimal solution (maximum money that can be made). Complexity is $O(nT)$

Given all the values in the two dimensional $\text{opt}()$ array, the optimal set of activities can be found in $O(n)$

Overall complexity of the algorithm is $O(nT)$ which is pseudopolynomial.

5) 20 pts

A polygon is called convex if all of its internal angles are less than 180° and none of the edges cross each other. We represent a convex polygon as an array V with n elements, where each element represents a vertex of the polygon in the form of a coordinate pair (x, y) . We are told that $V[1]$ is the vertex with the least x coordinate and that the vertices $V[1], V[2], \dots, V[n]$ are ordered counter-clockwise. Assuming that the x coordinates (and the y coordinates) of the vertices are all distinct, do the following.

Give a divide and conquer algorithm to find the vertex with the largest x coordinate in $O(\log n)$ time.

Solution:

Since $V[1]$ is known to be the vertex with the minimum x -coordinate (leftmost point), moving counter-clockwise to complete a cycle must first increase the x -coordinates and then after reaching a maximum (rightmost point), should decrease the x -coordinate back to that of $V[1]$. To see this more formally, we claim that there cannot be two distinct local maxima in the x -axis projection of the counter-clockwise tour. For the sake of contradiction, assume otherwise. Then there exists a vertical line that would intersect the boundary of the polygon at more than two points. This is impossible by convexity of the polygon since the line segments between the intersection points must lie completely in the interior of the polygon, thus contradicting our assumption. Thus, $V_x[1 : n]$ is a unimodal array, and the first part of this question is synonymous with detecting the location of the maximum element in this array.

Consider the following algorithm:

(a) If $n = 1$, return $V_x[1]$.

(b) If $n = 2$, return $\max\{V_x[1], V_x[2]\}$.

(c) $k = \lceil \frac{n}{2} \rceil$.

(d) If $V_x[k] > V_x[k-1]$ and $V_x[k] > V_x[k+1]$, then return $V_x[k]$.

(e) If $V_x[k] < V_x[k-1]$ then call the algorithm recursively on $V_x[1 : k-1]$, else call the algorithm recursively on $V_x[k+1 : n]$.

Complexity: If $T(n)$ is the running time on an input of size n , then beside a constant number of comparisons, the algorithm is called recursively on at most one of $V_x[1 : k-1]$ (size = $k-1 = \lceil \frac{n}{2} \rceil - 1 \leq \lfloor \frac{n}{2} \rfloor$) or $V_x[k+1 : n]$ (size = $n-k = n - \lceil \frac{n}{2} \rceil = \lfloor \frac{n}{2} \rfloor$). Therefore, $T(n) \leq T(\lfloor \frac{n}{2} \rfloor) + \theta(1)$.

Assuming n to be a power of 2, this recurrence simplifies to $T(n) \leq T(n/2) + \theta(1)$ and invoking Master's Theorem gives $T(n) = O(\log n)$.

Additional Space

RCS570 Fall 2017: Analysis of Algorithms Exam II

	Points
Problem 1	20
Problem 2	20
Problem 3	15
Problem 4	15
Problem 5	15
Problem 6	15
Total	100

Instructions:

1. This is a 2-hr exam. Closed book and notes
2. If a description to an algorithm or a proof is required please limit your description or proof to within 150 words, preferably not exceeding the space allotted for that question.
3. No space other than the pages in the exam booklet will be scanned for grading.
4. If you require an additional page for a question, you can use the extra page provided within this booklet. However please indicate clearly that you are continuing the solution on the additional page.
5. Do not detach any sheets from the booklet. Detached sheets will not be scanned.
6. If using a pencil to write the answers, make sure you apply enough pressure so your answers are readable in the scanned copy of your exam.
7. Do not write your answers in cursive scripts.

1) 20 pts

Mark the following statements as **TRUE** or **FALSE**. No need to provide any justification.

[**TRUE**/FALSE]

It is possible for a dynamic programming algorithm to have an exponential running time.

[**TRUE**/FALSE]

In a connected, directed graph with positive edge weights, the Bellman-Ford algorithm runs asymptotically faster than the Dijkstra algorithm.

[**TRUE** /FALSE]

There exist some problems that can be solved by dynamic programming, but cannot be solved by greedy algorithm.

[**TRUE** /FALSE]

The Floyd-Warshall algorithm is asymptotically faster than running the Bellman-Ford algorithm from each vertex.

[**TRUE** /FALSE]

If we have a dynamic programming algorithm with n^2 subproblems, it is possible that the space usage could be $O(n)$.

[**TRUE** /FALSE]

The Ford-Fulkerson algorithm solves the maximum bipartite matching problem in polynomial time.

[**TRUE** /FALSE]

Given a solution to a max-flow problem, that includes the final residual graph G_f . We can verify in a *linear* time that the solution does indeed give a maximum flow.

[**TRUE** /FALSE]

In a flow network, a flow value is upper-bounded by a cut capacity.

[**TRUE**/FALSE]

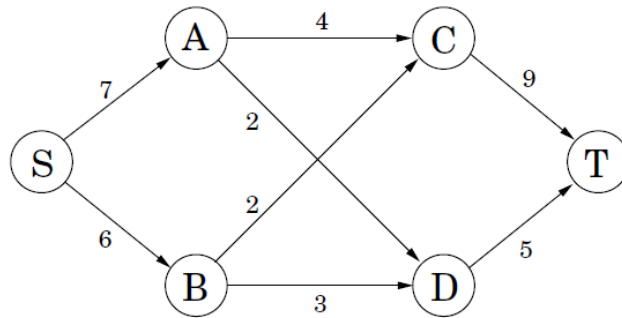
In a flow network, a min-cut is always unique.

[**TRUE**/FALSE]

A maximum flow in an integer capacity graph must have an integer flow on each edge.

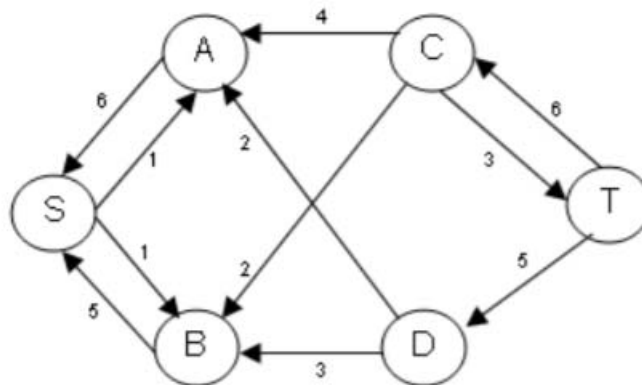
2) 20 pts.

You are given the following graph G . Each edge is labeled with the capacity of that edge.



- a) Find a max-flow in G using the Ford-Fulkerson algorithm. Draw the residual graph G_f corresponding to the max flow. You do not need to show all intermediate steps. (10 pts)

solution



Grading Rubrics:

Residual graph: 10 points in total

For each edge in the residual graph, any wrong number marked, wrong direction, or missing will result in losing 1 point.

The total points you lose is equal to the number of edges on which you make any mistake shown above.

b) Find the max-flow value and a min-cut. (6 pts)

Solution: $f = 11$, cut: ($\{S, A, B\}$, $\{C, D, T\}$)

Grading Rubrics:

Max-flow value and min-cut: 6 points in total

Max-flow: 2 points.

- If you give the wrong value, you lose the 2 points.

Min-cut: 4 points

- If your solution forms a cut but not a min-cut for the graph, you lose 3 points
- If your solution does not even form a cut, you lose all the 4 points

c) Prove or disprove that increasing the capacity of an edge that belongs to a min cut will always result in increasing the maximum flow. (4 pts)

Solution: increasing (B,D) by one won't increase the max-flow

Grading Rubrics:

Prove or Disprove: 4 points in total

- If you judge it "True", but give a structural complete "proof". You get at most 1 point
- If you judge it "False", you get 2 points.
- If your counter example is correct, you get the rest 2 points.

Popular mistake: a number of students try to disprove it by showing that if the min-cut in the original graph is non-unique, then it is possible to find an edge in one min-cut set, such that increasing the capacity of this does not result in max-flow increase.

But they did not do the following thing:

The existence of the network with multiple min-cuts needs to be proved, though it seems to be obvious. The most straightforward way to prove the existence is to give an example-network that has multiple min-cuts. Then it turns out to be giving a counter example for the original question statement.

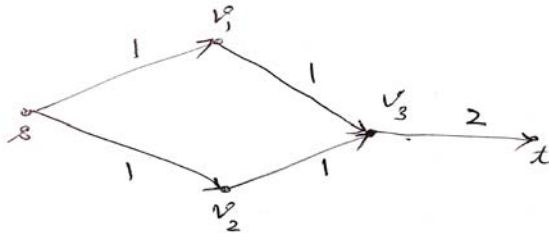
3) 15 pts.

Given a flow network with the source s and the sink t , and positive integer edge capacities c . Let (S,T) be a minimum cut. Prove or disprove the following statement:
If we increase the capacity of every edge by 1, then (S,T) still be a minimum cut.

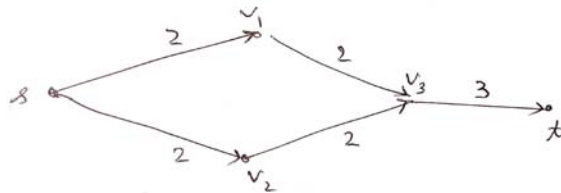
False. Create a counterexample.

An instance of a counter-example:

Initially, a (S,T) min cut is $S : \{s, v_1, v_2\}$ and $T : \{v_3, t\}$



After increasing capacity of every edge by 1, (S,T) is no longer a min-cut. We now have $S' : \{s, v_1, v_2, v_3\}$ and $T' : \{t\}$



Grading rubric:

If you try to prove the statement: -15

For an incorrect counter-example: -9

No credit for simply stating true or false.

4) 15 pts.

Given an unlimited supply of coins of denominations $d_1, d_2, \dots, d_n$, we wish to make change for an amount C . This might not be always possible. Your goal is to verify if it is possible to make such change. Design an algorithm by *reduction* to the knapsack problem.

a) Describe reduction. What is the knapsack capacity, values and weights of the items? (10 pts.)

Capacity is C . (2 points)
values = weights = d_k . (4points)

- You need to recognize that the problem should be modeled based on the unbounded knapsack problem with description of the reduction: (5 points)
- Explanation of the verification criteria (4 points)
 $\text{Opt}(j)$: the maximum change equal or less than j that can be achieved with $d_1, d_2, \dots, d_n$
 $\text{Opt}(0)=0$
 $\text{Opt}(j) = \max[\text{opt}(j-d_i)+d_i]$ for $d_i \leq j$
If we obtain $\text{Opt}(C) = C$, it means the change is possible.

b) Compute the runtime of your verification algorithm. (5 pts)

$O(nC)$ (3 points), Explanation (2 points).

5) 15 pts.

You are considering opening a series of electrical vehicle charging stations along Pacific Coast Highway (PCH). There are n possible locations along the highway, and the distance from the start to location k is $d_k \geq 0$, where $k = 1, 2, \dots, n$. You may assume that $d_i \leq d_k$ for $i \leq k$. There are two important constraints:

- 1) at each location k you can open only one charging station with the expected profit p_k , $k = 1, 2, \dots, n$.
- 2) you must open at least one charging station along the whole highway.
- 3) any two stations should be at least M miles apart.

Give a DP algorithm to find the maximum expected total profit subject to the given constraints.

a) Define (in plain English) subproblems to be solved. (3 pts)

Let $\text{OPT}(k)$ be the maximum expected profit which you can obtain from locations $1, 2, \dots, k$.

Rubrics

Any other definition is okay as long as it will recursively solve subproblems

b) Write the recurrence relation for subproblems. (6 pts)

$$\text{OPT}(k) = \max \{ \text{OPT}(k-1), p_k + \text{OPT}(f(k)) \}$$

where $f(k)$ finds the largest index j such that $d_j \leq d_k - M$, when such j doesn't exist, $f(k)=0$

Base cases:

$$\text{OPT}(1) = p_1$$

$$\text{OPT}(0) = 0$$

Rubrics:

Error in recursion -2pts, if multiple errors, deduction adds up

No base cases: -2pts

Missing base cases: -1pts

Using variables without definition or explanation: -2pts

Overloading variables (re-defining n , p , k , or M): -2pts

c) Compute the runtime of the above DP algorithm in terms of n . (3 pts)

algorithm solves n subproblems; each subproblem requires finding an index $f(k)$ which can be done in time $O(\log n)$ by binary search.
Hence, the running time is $O(n \log n)$.

Rubric:

$O(n^2)$ is regarded as okay

$O(d_n)$ pseudo-polynomial is also okay if the recursion goes over all d_n values

$O(n)$ is also okay

$O(n^3)$, $O(nM)$, $O(kn)$: no credit

d) How can you optimize the algorithm to have $O(n)$ runtime? (3 pts)

Preprocess distances $r_i = d_i - M$. Merge d-list with r-list.

Rubric

Claiming “optimal solution is already found in c” only gets credit when explanation about pre-process is described in either part b or c.

Without proper explanation (e.g. assume we have, or we can do): no credit

Keeping an array of max profits: no credit, finding the index that is closest to the installed station with M distance away is the bottleneck, which requires pre-processing.

6) 15 pts.

A group of traders are leaving Switzerland, and need to convert their Francs into various international currencies. There are n traders $t_1, t_2, \dots, t_n$ and m currencies $c_1, c_2, \dots, c_m$. Trader t_k has F_k Francs to convert. For each currency c_j , the bank can convert at most B_j Francs to c_j . Trader t_k is willing to trade as much as S_{kj} of his Francs for currency c_j . (For example, a trader with 1000 Francs might be willing to convert up to 200 of his Francs for USD, up to 500 of his Francs for Japanese's Yen, and up to 200 of his Francs for Euros). Assuming that all traders give their requests to the bank at the same time, describe an algorithm that the bank can use to satisfy the requests (if it can).

a) Describe how to construct a flow network to solve this problem, including the description of nodes, edges, edge directions and their capacities. (8 pts)

Bipartite graph: one partition traders $t_1, t_2, \dots, t_n$. Other, available currency, $c_1, c_2, \dots, c_m$.

Connect t_k to c_j with the capacity S_{kj}

Connect source to traders with the capacity F_k .

Connect available currency c_j to the sink with the capacity B_j .

Rubrics:

- Didn't include supersource (-1 point)
- Didn't include traders nodes (-1 point)
- Didn't include currencies nodes (-1 point)
- Didn't include supersink (-1 point)
- Didn't include edge direction (-1 point)
- Assigned no/wrong capacity on edges between source & traders (-1 point)
- Assigned no/wrong capacity on edges between traders & currencies (-1 point)
- Assigned no/wrong capacity on edges between currencies & sink (-1 point)

b) Describe on what condition the bank can satisfy all requests. (4 pts)

If there is a flow f in the network with $|f| = F_1 + \dots + F_n$, then all traders are able to convert their currencies.

Rubrics:

- Wrong condition (-4 points): Unless you explicitly included $|f| = F_1 + \dots + F_n$, no partial points were given for this subproblem.
- In addition to correct answer, added additional condition which is wrong (-2 points)
- No partial points were given for conditions that satisfy only some of the requests, not all requests.
- Note that $\sum S_{kj}$ and $\sum B_j$ can be larger than $\sum F_k$ in some cases where bank satisfy all requests.
- Note that $\sum S_{kj}$ can be larger than $\sum B_j$ in some cases where bank satisfy all requests.
- It is possible that $\sum S_{kj}$ or $\sum B_j$ is smaller than $\sum F_k$. In these cases, bank can never satisfy all requests because max flow will be smaller than $\sum F_k$.

- c) Assume that you execute the Ford-Fulkerson algorithm on the flow network graph in part a), what would be the runtime of your algorithm? (3 pts)

$O(n \cdot m \cdot |f|)$

Rubrics:

- Flow($|f|$) was not included (-1 point)
- Wrong description (-1 point)
- Computation is incorrect (-1 point)
- Notation error (-1 point)
- Missing big O notation (-1 point)
- Used big Theta notation instead of big O notation (-1 point)
- Wrong runtime complexity (-3 points)
- No runtime complexity is given (-3 points)

CS570 Fall 2021: Analysis of Algorithms Exam II

	Points		Points
Problem 1	20	Problem 4	20
Problem 2	20	Problem 5	20
Problem 3	20		
	Total	100	

Instructions:

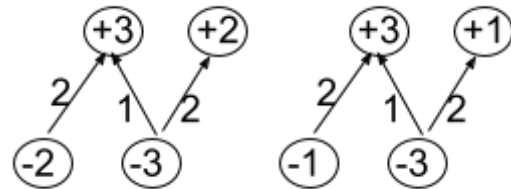
1. This is a 2-hr exam. Open book and notes and internet access. But no internet communications through social media, chat, or any other form is allowed.
2. If a description to an algorithm or a proof is required, please limit your description or proof to within 150 words, preferably not exceeding the space allotted for that question.
3. No space other than the pages in the exam booklet will be scanned for grading.
4. If you require an additional page for a question, you can use the extra page provided within this booklet. However please indicate clearly that you are continuing the solution on the additional page.
5. Do not detach any sheets from the booklet. Detached sheets will not be scanned.
6. If using a pencil to write the answers, make sure you apply enough pressure, so your answers are readable in the scanned copy of your exam.
7. Do not write your answers in cursive scripts.
8. This exam is printed double sided. Check and use the back of each page.

True/False Questions

Network flow (2 pts)

1. We are guaranteed to find max flow using Ford-Fulkerson as long as our edge capacities are all positive.
False. Not guaranteed to terminate or converge to max flow for arbitrary non-integral capacities.
2. If a flow network G contains an edge that goes directly from source S to sink T , then this edge will always be saturated due to any max flow in G .
True. This edge is a part of every min-cut, thus must be saturated at max flow.
3. The capacity of an s - t cut in a Flow Network G could be greater than the value of max flow in G .
True. There could always be a cut of size greater than that of a min-cut (= max flow value).
4. In a Circulation Network with no lower bound constraints and a feasible circulation F , if we decrease the demand at a sink node by one unit and decrease the supply at a source node by one, we will still have a feasible circulation in the resulting Circulation Network.

False. Consider the network shown on the side with capacities and demands, initially as on the left, and the changed ones as on the right. Initially there's a feasible circulation (obtained by saturating all the edges). After the changed demands, there is no feasible circulation.



5. If f and f' are two feasible s - t flows in a Flow Network such that $|f'| > |f|$, then there is always a feasible s - t flow in the network with value $|f'| - |f|$.
True. If f is a feasible flow, there is always a feasible flow of any lower value (this is a simple flow network, not a circulation with lower bounds/demands).

Dynamic Programming (2 pts)

1. The memory space required for any dynamic programming algorithm with n^2 unique subproblems is $\Omega(n^2)$.
False. We have seen examples where only very few subproblems (and not all) need to be stored at any point during the DP computation of all the sub-problems. In such cases, the memory needed can be much less.
2. The 0-1 knapsack problem can be solved using dynamic programming in $O(N * V)$ runtime, where N is the number of items and V is the **sum of the weights of all items**.
True.
3. In a 0/1 knapsack problem with n items, suppose the value of the optimal solution for all unique subproblems has been found. If one adds a new item to the list now, one must re-compute all values of the optimal solutions for all unique subproblems in order to find the value of the optimal solution for the $n+1$ items.
False. We can simply compute $\text{Opt}(W, n+1) = \text{Opt}(W, n) + \text{Opt}(W - W_{n+1}, n)$ with problems on the right already computed.

Dynamic Programming (4 pts)

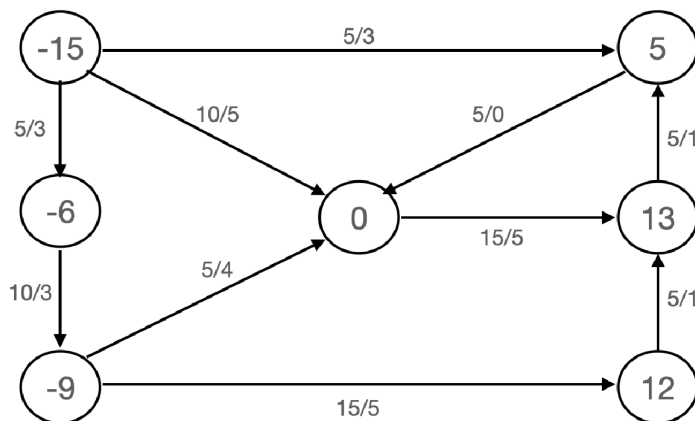
4. Recall the solution to the 0/1 knapsack problem presented in lecture. If our objective were to only find the **value of the optimal solution** (and not the actual set of items in the optimal solution), $O(n)$ memory space will be sufficient to solve this problem.

False. To compute all the values $\text{Opt}(w, i)$ for all w , for a given i , we need all the values $\text{Opt}(w, i-1)$ for all w . Thus, we need to store $O(W)$ memory, which is not necessarily $O(n)$.

Network Flow Problem 1

(20 pts)

In the network G below, the demand values are shown on vertices (supply value if negative). Lower bounds on flow and edge capacities are shown as (capacity/lower bound) for each edge. Determine if there is a feasible circulation in this graph. You need to show all your steps.

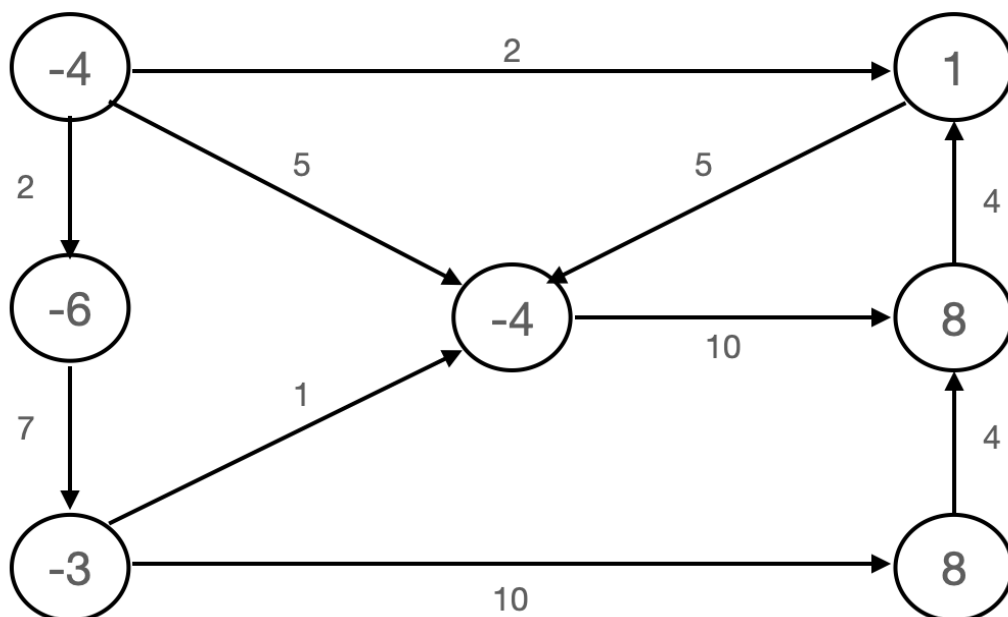


a. Reduce the Feasible Circulation with Lower Bounds problem to a Feasible Circulation problem without lower bounds. (8 pts)

Draw G'

If demand is correct, get 4 pts

If flow on edges are correct, get 4 pts



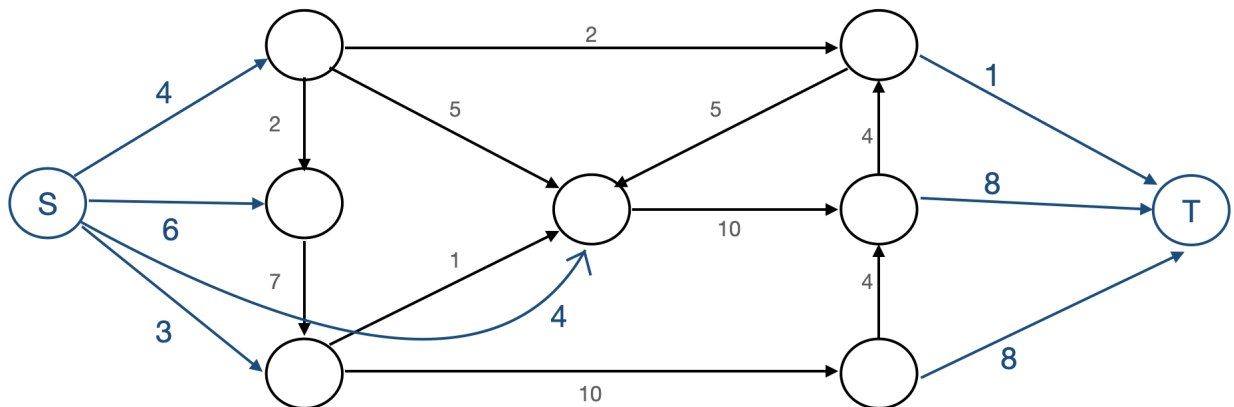
b. Reduce the Feasible Circulation problem obtained in *part a* to a Maximum Flow problem in a Flow Network.(8 pts)

Add S and T (2 pts)

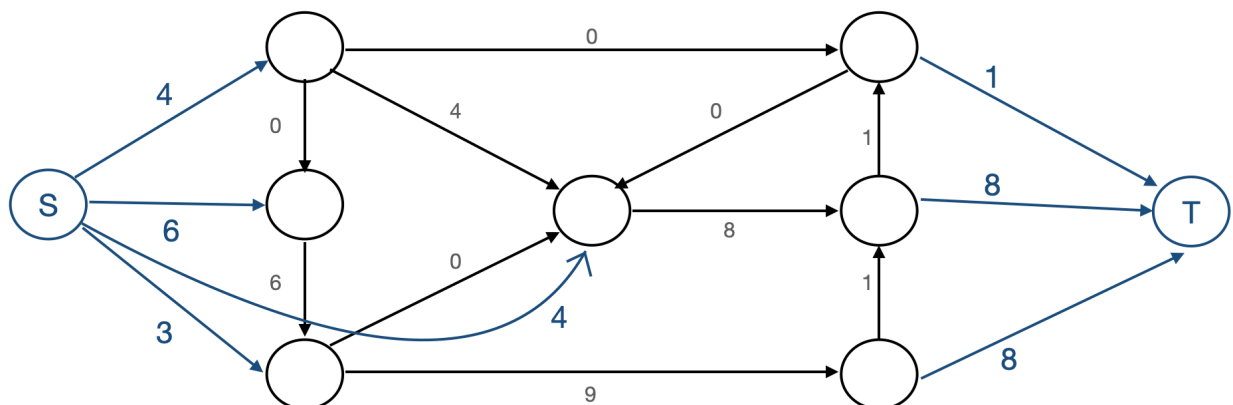
Connect S with correct nodes (2 pts)

Connect T with correct nodes (2 pts)

Give correct capacities to edges (2pts)



c. Using the solution to the resulting Max Flow problem explain whether there is a Feasible Circulation in G. (4 pts)

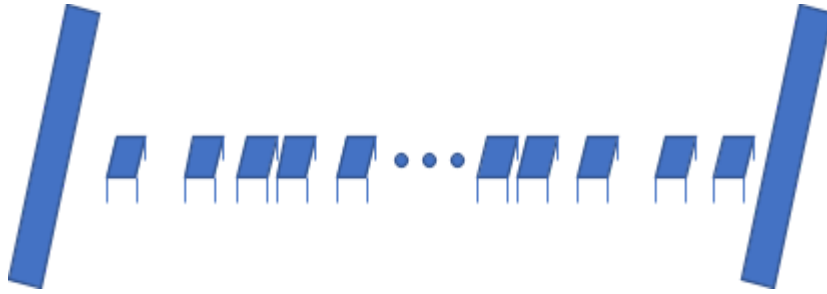


This is the max flow in Flow network. (2 pts)

Since the flow saturates all edges from S, so there is feasible circulation in G. (2 pts)

Network Flow Problem 2 (20 pts)

A number of people have gotten themselves involved in a very dangerous game called the octopus game. At this stage of the game, they need to pass a river. The river is 100 feet wide, but contestants can only jump k feet at most, where k is obviously less than 100. To help contestants cross the bridge there are platforms placed along a straight line at integer distances from one end of the river to the other end at various distances where the distance between any two consecutive platform is less than k . But the problem is that each platform can only be used once. If another contestant tries to use it for the second time it will break.



- a) Design a network flow based solution to determine the maximum number of contestants that can safely cross this river and live to play in the next stage of the game. (14 pts)

Solution 3 a)

The construction of the network can be done as follows:

1. Create a source 'S' and sink node 'T' and create nodes that would represent the platforms (lets assume we call these nodes $\{A, \dots, N\}$)
2. For every platform node $\{A, \dots, N\}$ create a dummy node $\{A', B', \dots, N'\}$ respectively. Connect all the platform nodes $\{A, \dots, N\}$ to the corresponding dummy nodes $\{A', \dots, N'\}$ so that the edge $A \rightarrow A'$ represents platform A and so on. Set the capacity to 1 for all these edges. This part is the most crucial in the construction to make sure that a platform can not be used by more than one contestants.
3. Connect the source 'S' to the platform nodes that are at max k feet distance from it. The capacity of any such edge could be set to anything ≥ 1 . This is to make sure that all the inflow to the nodes is at platform node $\{A, B, \dots, N\}$.
4. Connect the dummy nodes $\{A', B', \dots, N'\}$ to all the platform nodes (or the sink) that are at max k feet distance from it. HOWEVER, it suffices to add such edges in only the forward direction. This also makes sure the outflow at each platform is from the dummy nodes $\{A', B', \dots, N'\}$

Final answer: Once the network is constructed, run a max flow algorithm on it (say FF) and the max flow value of the network obtained will be the maximum number of contestants that can cross the river.

Rubrics 3a:

- 2 points for each incorrect/missing capacity/node/edge
- -2 pts: if the final answer is missing
- AT MOST half the points if the fundamental structure of the network is wrong (partial credit subjective to how close the proposed solution is)

b) Prove that your solution is correct (6 pts)

Claim : The max flow of the network will give the number of contestants that can get across the river

Forward Claim : If there is a flow of value V , we can send V people across

Proof : Since $k < 100$, any S-T path must pass via some platform (say p), and thereby has a bottleneck of 1 due to the edge $p \rightarrow p'$. Thus any s-t path can have a flow of at most 1. Thus, if the flow value is V , there will be V paths each having a flow of 1. Further, these cannot share any edge $p \rightarrow p'$ due to its capacity of 1. Thus, we can assign each such path to a contestant and none of them will use the same platform, thus allowing us to send V people safely.

Backward Claim : If we can send V people across, we can have a flow of value V

Proof : For 1 person crossing the river, we get a path going from s to t such that the person jumps at most k feet at a time. These V paths must be node-disjoint, since no platforms can be repeated. If we send a flow of 1 unit down each corresponding path in the network, we won't exceed any capacity constraints and this flow will have a value of V .

Rubrics 3b:

- -2 pts: If the claim is incorrect
- -2 pts: If forward claim is not satisfactory/incorrect.
- -2 pts: If backward claim is not satisfactory/incorrect.

Dynamic Programming Problem 1 (20 pts)

Tommy has just joined the boy scouts and he's eager to earn some badges. He's got summer holidays coming up which will last for N days. Each day, he can earn points by doing any one activity among *hiking*, *swimming*, or *camping*. The camp instructor has posted the amount of points each activity can earn you on each day. For example, on the i th day, camping is worth $c[i]$ points, hiking $h[i]$ points and swimming $s[i]$ points. However, Tommy gets bored doing the same activity really easily. He cannot do the same activity two or more days in a row.

Based on this information, devise a Dynamic Programming algorithm to maximize the number of points Tommy can earn. You may assume that all point values are positive, and remember he can only do one activity per day.

a) Define (in plain English) subproblems to be solved. (4 pts)

$OPT_{i,j}$ = the maximum number of points Tommy can earn from the start until the i^{th} day, given he does activity j on the i^{th} day.

- -2 points if it is mentioned: "points earned **ON** the i th day" instead of **until** the i th day.
- -4 points for incorrectly defined subproblems

OR

$OPT_{i,j}$ = the maximum number of points Tommy can earn from the i^{th} day, until the end given he does activity j on the i^{th} day.

b) Write a recurrence relation for the subproblems (6 pts)

$$OPT_{i,j} = \max(OPT_{i-1,k} + act[j][i] \text{ where } j \neq k, j, k \in \{c, h, s\}, act = \{c, h, s\})$$

In the second variant, $i-1$ is replaced by $i+1$ in each of the terms on the RHS

c) Using the recurrence formula in part b, write pseudocode using iteration to compute the maximum number of points Tommy can earn. (5 pts)

Make sure you specify

- base cases and their values (2 pts)
- where the final answer can be found (e.g. $opt(n)$, or $opt(0,n)$, etc.) (1 pt)

For variant 1:

```
function POINTS_EARNED(N, c, h, s):  
    act = [c, h, s]  
    OPT = [[0, 0, 0] N times]  
    OPT[0][0] = c[0], OPT[0][1] = h[0], OPT[0][2] = s[0]
```

```

    for i in 1, 2, ..., N - 1:
        for j in 0, 1, 2:
            for k in 0, 1, 2:
                if j != k:
                    OPT[i][j] = max(OPT[i][j], OPT[i - 1][k] +
act[j][i])
            END-FOR
        END-FOR
    END-FOR

return max(OPT[N - 1])

```

-2 points for not specifying base cases/values, or for incorrect base cases/values
-1 point for each mistake in pseudocode
-1 point for incorrect return value or incorrect final answer

d) What is the complexity of your solution? (1 pt)
Is this an efficient solution? (1 pt)

$O(N)$
Yes.

1 point for correct runtime complexity analysis of **YOUR** solution. 0 points if analysis is incorrect.
0 points for saying it is not efficient.

Dynamic Programming Problem 2 (20 pts)

Assume a truck with capacity W is loading. There are n package types with different weights, i.e. $[w_1, w_2, \dots, w_n]$, and all the weights are integers. Packages of the same type have the same weight. The company's rule requires that the truck needs to take packages with exactly weight W to maximize profit, but the workers like to save their energies for after work activities and want to load as few packages as possible. Assuming that there are infinite packages for each package type and that there will always be combinations of packages with a total weight of W , design an algorithm to find out the minimum number of packages the workers need to load.

a) Define (in plain English) subproblems to be solved. (4 pts)

$OPT(w,k)$ = min packages needed to make up a capacity of exactly w , by considering only the first k package types

OR

Same except, considering package types k onwards up to n (add details below)

b) Write a recurrence relation for the subproblems (6 pts)

If $w \geq w_k$

$OPT(w,k) = \min \{ 1 + OPT(w - w_k, k), OPT(w, k-1) \}$

Else

$OPT(w,k) = OPT(w, k-1)$

For variant 2, replace $k-1$ by $k+1$.

c) Using the recurrence formula in part b, write pseudocode using iteration to compute the minimum number of packages to meet the objective. (5 pts)

Make sure you specify

1. Initialize for base cases (mentioned below for clarity). Initialize rest to infinity
2. For $w = 0$ to W
 For $k = 0$ to n //can switch the inner-outer loops
 If not base case: call recurrence
3. Return ans (mentioned below for clarity)

- base cases and their values (2 pts)

$OPT(w,0) = \text{inf for all } w > 0$

$OPT(0,0) = 0$

- where the final answer can be found (e.g. $opt(n)$, or $opt(0,n)$, etc.) (1 pt)

$OPT(W,n)$

d) What is the complexity of your solution? (1 pt)

Is this an efficient solution? (1 pt)

$O(nW)$ pseudo-polynomial run time
This is not an efficient solution

Solution 2:

a) Define (in plain English) subproblems to be solved. (4 pts)

$OPT(w,k)$ = min packages needed to make up a capacity of exactly w , by considering only the first k packages, AND selecting package k for sure.

b) Write a recurrence relation for the subproblems (6 pts)

The recurrence captures all cases of j where j was the highest index used before the k^{th} one:

If $w \geq w_k$

$OPT(w,k) = 1 + \min_{j=0 \text{ to } k} OPT(w - w_k, j)$

Else

$OPT(w,k) = \text{infinity}$

(No penalty for missing the latter case IF the base cases in 'c)' include $w < 0$.)

c) Using the recurrence formula in part b, write pseudocode using iteration to compute the minimum number of packages to meet the objective. (5 pts)

1. Initialize for base cases (mentioned below for clarity). Initialize rest to infinity

2. For $w = 0$ to W

For $k = 0$ to n //can switch the inner-outer loops

If (not base case): //call recurrence

For $j = 0$ to k

$OPT(w,k) = \min\{OPT(w,k), 1 + OPT(w - w_k, j)\}$

3. Return ans (mentioned below for clarity)

Make sure you specify

- base cases and their values (2 pts)

$OPT(w,0) = \text{inf}$ for all $w > 0$

$OPT(0,0) = 0$

$OPT(w,k) = \text{infinity}$ for $w < 0$ and all k . (Not required if the recurrence has the second case to ensure w never takes negative values)

- where the final answer can be found (e.g. $\text{opt}(n)$, or $\text{opt}(0,n)$, etc.) (1 pt)

$\text{Max}_k OPT(W, k)$

d) What is the complexity of your solution? (1 pt)

Is this an efficient solution? (1 pt)

$O(n^2W)$ pseudo-polynomial run time
This is not an efficient solution

Solution3: (1-D) subproblems

a) Define (in plain English) subproblems to be solved. (4 pts)

$OPT(w) = \min$ packages needed to make up a capacity of exactly w , (by considering all packages)

b) Write a recurrence relation for the subproblems (6 pts)

$$OPT(w) = 1 + \min_{\{j=1 \text{ to } n, w \geq w_j\}} OPT(w - w_j)$$

(No penalty for missing the second condition under $\min$. IF the base cases in 'c)' include $w < 0$.)

c) Using the recurrence formula in part b, write pseudocode using iteration to compute the minimum number of packages to meet the objective. (5 pts)

1. Initialize for base cases (mentioned below for clarity). Initialize rest to infinity
2. For $w = -W$ to W
 - If (not base case): //call recurrence
 - For $j = 1$ to n
 - $OPT(w) = \min\{OPT(w), 1 + OPT(w - w_j)\}$
3. Return ans (mentioned below for clarity)

Make sure you specify

- base cases and their values (2 pts)

$$OPT(0) = 0$$

$OPT(w) = \text{infinity}$ for $w < 0$ (Not required if the recurrence has the second case to ensure w never takes negative values)

- where the final answer can be found (e.g. $opt(n)$, or $opt(0,n)$, etc.) (1 pt)

$OPT(W)$

CS570 Fall 2021: Analysis of Algorithms Exam II

	Points		Points
Problem 1	20	Problem 4	20
Problem 2	20	Problem 5	20
Problem 3	20		
	Total	100	

Instructions:

1. This is a 2-hr exam. Open book and notes and internet access. But no internet communications through social media, chat, or any other form is allowed.
2. If a description to an algorithm or a proof is required, please limit your description or proof to within 150 words, preferably not exceeding the space allotted for that question.
3. No space other than the pages in the exam booklet will be scanned for grading.
4. If you require an additional page for a question, you can use the extra page provided within this booklet. However please indicate clearly that you are continuing the solution on the additional page.
5. Do not detach any sheets from the booklet. Detached sheets will not be scanned.
6. If using a pencil to write the answers, make sure you apply enough pressure, so your answers are readable in the scanned copy of your exam.
7. Do not write your answers in cursive scripts.
8. This exam is printed double sided. Check and use the back of each page.

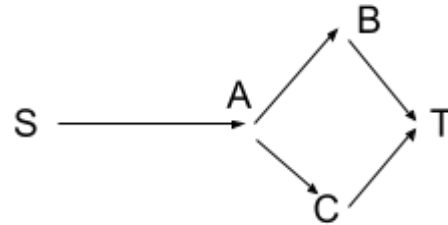
1) Mark the following statements as **TRUE** or **FALSE**. No need to provide justification.

[TRUE/FALSE] (2 pts)

If all edge capacities in a Flow Network are integer multiples of 5 then considering any max flow F in this network, flow over each edge due to F must be an integer multiple of 5.

False. There will always exist a flow with flow over each edge being a multiple of 5, but this may not be true of ANY flow.

Counter - example as shown on the side. Let all capacities be 5. Let $f(SA) = 5$, $f(AB) = f(AC) = 2$, $f(BT) = f(CT) = 3$. Then, f is a max flow.

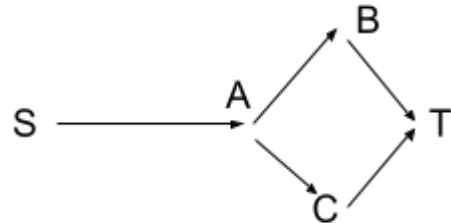


[TRUE/FALSE] (2 pts)

In a Flow Network G , if the capacity of any edge on a min cut is increased by 1 unit, then the value of max flow in that network will also increase by 1 unit.

Note: an edge on the min cut (A, B) refers to an edge that carries flow out of A (where the source S is) and into B (where the sink T is).

False. As counter - example, consider the network as shown on the side. Let all capacities be 5. The max flow here is of value 5. Consider the cut having AB, AC as the cut-edges. Increasing one of these to a capacity of 6 still keeps max flow at 5.



[TRUE/FALSE] (2 pts)

In a Flow Network with maximum flow value $v(F) > 0$, and with more than one min cut, decreasing the capacity of an edge on any of the min cuts will reduce the value of max flow.

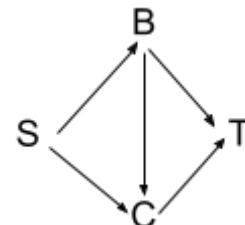
Note: an edge on the min cut (A, B) refers to an edge that carries flow out of A (where the source S is) and into B (where the sink T is).

True. Suppose the capacity of an edge in a min-cut (A,B) is decreased from c to c' . Initially, $v(F)$ must have been c . Now, the max flow must be bounded by the new size of cut (A,B) , i.e., c' , thus, it does decrease.

[TRUE/FALSE] (2 pts)

Consider the Ford Fulkerson algorithm and the residual graph used at each iteration. If the Flow Network has no cycles (i.e. the network is a directed acyclic graph), we will not need to include backward edges in the residual graph to achieve max flow.

False. Consider the counter-example shown on the side. This is a DAG (no cycles). Let all capacities be 5. Suppose FF chooses path $SBCT$ in the first step. If back-edges are not added, the residual graph will have T disconnected from S at this stage and terminate - at flow 5. Allowing the back edges however, we get path $SCBT$ as step 2 making the actual max flow of 10.



[TRUE/FALSE] (2 pts)

Suppose an edge e is not saturated due to max flow f in a Flow Network. Then increasing e 's capacity will not increase the max flow value of the network.

True. Since e is not saturated, e is not a cut-edge for any min-cut. Thus, increasing the capacity of e , does not affect the size of any min-cut and hence the max-flow value

remains the same.

[TRUE/FALSE] (2 pts)

The time complexity of any dynamic programming algorithm with n^2 unique subproblems is $\Omega(n^2)$.

True.

[TRUE/FALSE] (2 pts)

The Bellman-ford algorithm may not be able to find the shortest simple path in a graph with negative cost edges.

True.

[TRUE/FALSE] (2 pts)

If the running time of an algorithm can be represented as a polynomial in terms of the number of bits in the input, then the algorithm is considered to be efficient.

True.

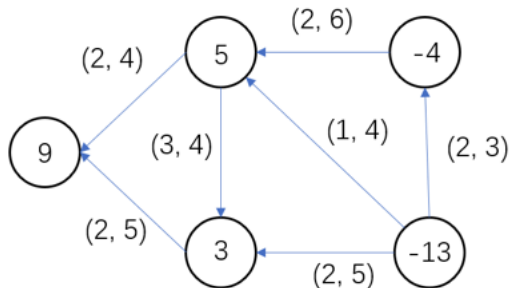
[TRUE/FALSE] (4 pts)

Recall the coin change problem from lecture where we are trying to pay amount m using the minimum number of coins. We presented two attempts to solve this problem in class, one based on the greedy approach and one using dynamic programming. If coin denominations are limited to 1, 3, and 4 these two approaches will result in the same number of coins to pay any amount m .

False. DP always gives the optimal solution. Greedy does not for $m = 6$.

2) 20 pts

In the network G below, the demand values are shown on vertices (supply value if negative). Lower bounds on flow and edge capacities are shown as (lower bound, capacity) for each edge. Determine if there is a feasible circulation in this graph. You need to show all your steps.



- a. Reduce the Feasible Circulation with Lower Bounds problem to a Feasible Circulation problem without lower bounds. (8 pts)

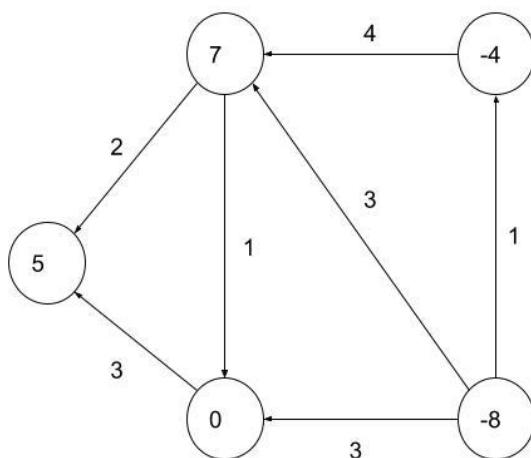
Solution:

Steps to follow :- 1) Assign flows to edges to satisfy lower bounds.

2) At each node v , $L_v = (f_{in} - f_{out})$, followed by $D' = D - L_v$

3) At each edge, $cap = cap - LB$

Resultant network:



Rubrics:

(4 Pts) : If all edge values: (Upper_bound - Lower_bound) is correctly calculated

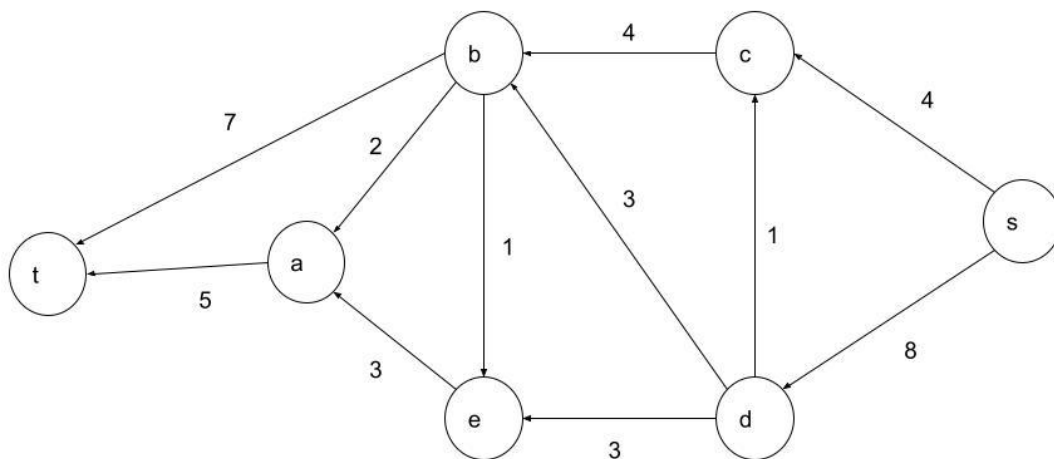
(4 Pts): If all Node Values ($D' = D - (f_{in} - f_{out})$) is correctly calculated

(-2 Pts) if no steps explained.

b. Reduce the Feasible Circulation problem obtained in *part a* to a Maximum Flow problem in a Flow Network. (8 pts)

Solution:

The Max-Flow graph is as follows:



Rubrics:

Add S and T (2 pts)

Connect S with correct nodes (2 pts)

Connect T with correct nodes (2 pts)

Give correct capacities to edges (2pts)

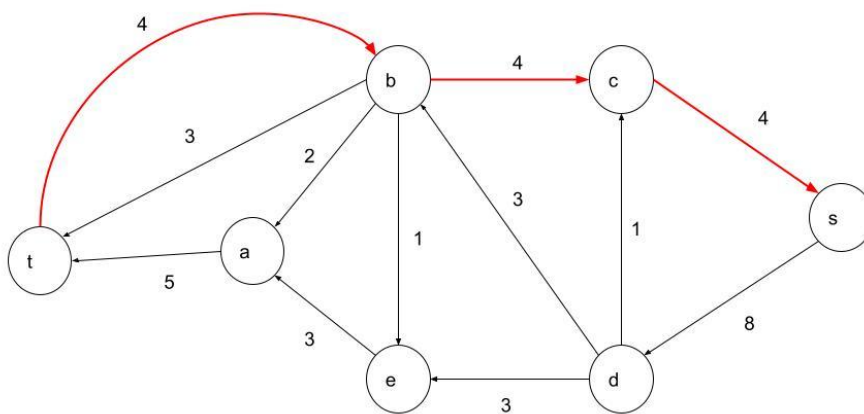
c. Using the solution to the resulting Max Flow problem explain whether there is a Feasible Circulation in G. (4 pts)

Solution:

Candidate Solution 1:

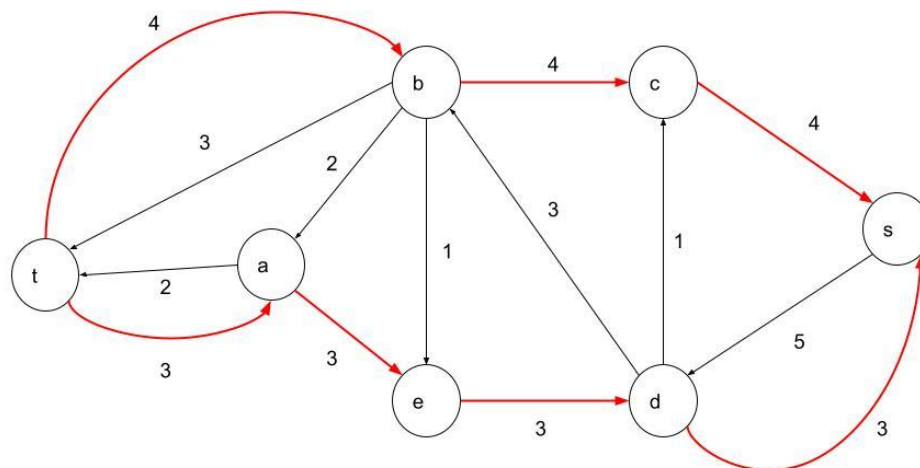
First Augmenting Path: $s \rightarrow c \rightarrow b \rightarrow t$ with flow = 4

Residual Graph 1:



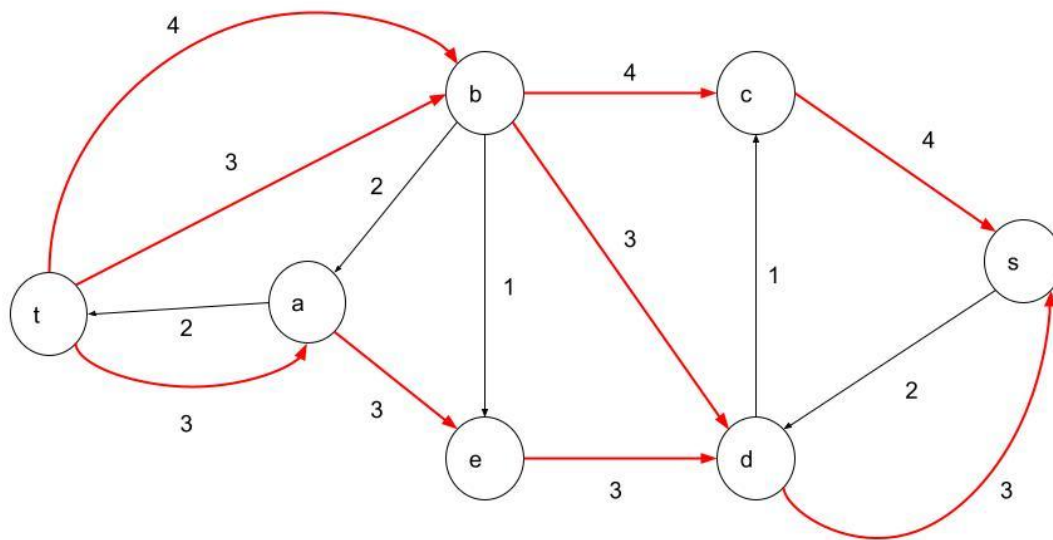
Second Augmenting Path: $s \rightarrow d \rightarrow e \rightarrow a \rightarrow t$ with flow = 3

Residual Graph 2:



Third Augmenting path: $s \rightarrow d \rightarrow b \rightarrow t$ with flow 3

Residual Graph 3:



Candidate Solution 2:

First Augmenting Path: $s \rightarrow c \rightarrow b \rightarrow t$ with flow = 4

Second Augmenting Path: $s \rightarrow d \rightarrow b \rightarrow a \rightarrow t$ with flow = 2

Third Augmenting Path: $s \rightarrow d \rightarrow b \rightarrow t$ with flow = 1

Fourth Augmenting Path: $s \rightarrow d \rightarrow e \rightarrow a \rightarrow t$ with flow = 3

Candidate Solution 3:

First Augmenting Path: $s \rightarrow d \rightarrow e \rightarrow a \rightarrow t$ with flow = 3

Second Augmenting Path: $s \rightarrow d \rightarrow b \rightarrow a \rightarrow t$ with flow = 2

Third Augmenting Path: $s \rightarrow d \rightarrow c \rightarrow b \rightarrow t$ with flow = 1

Fourth Augmenting Path: $s \rightarrow c \rightarrow b \rightarrow t$ with flow = 3

Fifth Augmenting Path: $s \rightarrow d \rightarrow b \rightarrow t$ with flow = 1

Max-Flow = 10

Since, the value of Max-Flow is less than the total demand value $D=12$, there is **No Feasible solution in the circulation network, and therefore there is no feasible circulation in the circulation with lower bounds network.**

Rubrics:

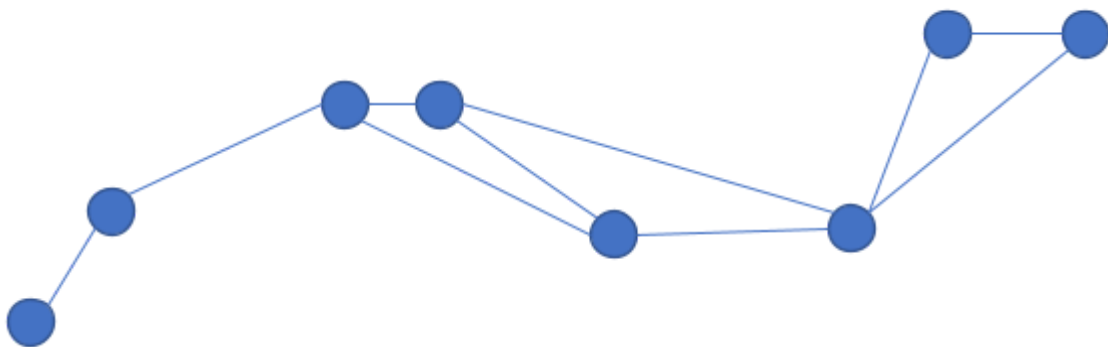
2 pts: Finding correct Max-Flow and presenting their appropriate residual graphs according to the sequence of augmenting paths that the student has chosen

1 pt: Correctly concluding “No Feasible Circulation”

1 pt: Reasoning behind “No Feasible Circulation”

3) 20 pts

In the last 570 exam, there were nearly a thousand students that were ready to take the exam in person at 8 different test locations (L1..L8). There were S_i students assigned to each room i . The copy center made some big mistakes and instead of delivering S_i papers they delivered R_i papers to each room i , where R_i was higher than S_i for some rooms and lower for some others. TAs then had to rush and come up with a solution to redistribute the excess papers at each test location to those test locations that had a shortage. And because the time was short, they ruled out sending papers directly between test locations that were far apart from each other on campus. But they did not rule out sending papers indirectly through other test locations. (For example, L1 and L3 in below graph were considered far apart from each other but papers could still be redistributed between them through L2). So with these considerations in mind, they ended up with a connected graph that looked like this:



- a) Assuming that the total number of papers delivered was at least equal to the total number of papers required, design a network flow based algorithm to determine how many papers had to be sent (and in which direction) on each edge in the above network in order to supply each room with the required papers. You need to describe exactly how you reduce this problem to a network flow problem. (14 pts)

Solution 3a):

Across all the candidate solutions below, the common steps initially are

1. Use the locations as nodes in the flow network.
2. Convert each undirected edge in the given graph to 2 directed edges in opposite directions in the flow network.
3. No constraints on how many papers MUST be carried along any such edge, so no lower bound. Capacity should be a suitable high value: Infinity (= conservative safest upper bound) OR Summation of S_i (= papers needed in total) OR Summation of R_i (= papers available in total) OR $\sum_{\{i \text{ where } S_i - R_i > 0\}} (S_i - R_i)$ (= sum of shortages of papers) OR $\sum_{\{i \text{ where } R_i - S_i > 0\}} (R_i - S_i)$ (= sum of surplus of papers). The flow on an edge between two locations can be at most any of these upper-bounds.

Candidate Solution 1: Circulation with lower bounds:

1. Add a source S and edges $S \rightarrow L_i$ for each location. Each such edge corresponds to the papers sent from the copy center. Since this is exactly R_i , set its LB/cap to R_i/R_i .
2. Add a sink T and edges $L_i \rightarrow T$ for each location. Each such edge corresponds to the papers finally retained at location L_i . This needs to be at least S_i , thus that's the LB. It

could be much more; a capacity of as low as $\max \{S_i, R_i\}$ works (a safe upper bound is simply a capacity of infinity)

3. Add a demand value of $\sum R_i$ at T and $-\sum R_i$ at S. Alternatively (instead of adding the demands at S,T), adding a $T \rightarrow S$ edge suffices. (This is not always equivalent, but works here since the net S-T flow value is fixed due to the LB/cap values of R_i/R_i on the edges out of S). The capacity of this back-edge has to be at least $\sum R_i$.

Solution 2: Relative to solution 1, The lower bounds on the edges from S could be removed by adjusting the capacities/demands around. Consequently,, we have no source, but do have a sink, and each L_i has a demand of ' $-R_i$ ' and they are all connected to the sink with demand $\sum R_i$

Solution 3: Relative to solution 2, , we can get rid of the lower bounds on edges $L_i \rightarrow T$. In this case, we have (In addition to the common steps laid out in the beginning) a demand of $S_i - R_i$ at location L_i and sink T with a demand (sum of R_i - sum of S_i). We have edges $L_i \rightarrow T$ of capacity $R_i - S_i$ wherever it is positive (but okay to add edges from all the $L_i \rightarrow T$ and okay to have bigger capacities).

Solution 4: The circulation network from Solution 3 can be further converted to a simple flow network by getting rid of the demands and adding a super source S^* and a super sink T^* . S^* must connect to all L_i where $R_i - S_i$ is positive, with as much edge capacity. Each L_i with positive $S_i - R_i$ connects to T^* with as much capacity and T connects to T^* with capacity (sum of R_i - sum of S_i). This network must have a max flow of $\sum_{\{i \text{ where } R_i - S_i > 0\}} (R_i - S_i)$.

Solution 5: A modified version of Solution 4 can get rid of T and all the edges coming in and out of it. This network must have a max flow of value $\sum_{\{i \text{ where } S_i - R_i > 0\}} (S_i - R_i)$ (note this crucial difference with respect to max flow value of Solution 4).

Solution 6: Create a source S, and connect $S \rightarrow L_i$ nodes with capacity R_i and create a sink T, and connect each $L_i \rightarrow T$ with capacity S_i . This network must have a max flow of value $\sum (S_i)$

Solution 7: This one has a different structure with two partitions (L and L') EACH having nodes for ALL 8 locations. Create S, and edges from $S \rightarrow L_i$ nodes with capacity R_i . Create a sink T, and connect $L'_i \rightarrow T$ with capacity S_i . Each L_i and its copy L'_i should have edges both ways. But for $i \neq j$, $L_i \rightarrow L'_j$ can be an edge in just this one direction. ALL these edges between L and L' must have the same high capacity as outlined in the beginning of this solution. This network must have a max flow of value $\sum (S_i)$

Final answer (i.e. paper redistribution scheme) required in each solution: Compute the feasible circulation (in solutions 1/2/3) or max flow (in solutions 4/5/6/7) F. From location L_i to an adjacent L_j send $F(L_i \rightarrow L_j) - F(L_j \rightarrow L_i)$ papers (or in the reverse direction, whichever gives a positive number).

Rubrics 3a:

- -2 points for each incorrect/missing demand/LB/capacity/node/edge

- -2 pts: if the final algorithm description of how to compute redistribution solution from the constructed network is missing.
- AT MOST half the points if the fundamental structure of the network is wrong (partial credit subjective to how close the proposed solution is)

a) Prove that your solution is correct (6 pts)

Solution 3b):

Here's the proof for solution 1 (Circulation without lower bounds)

Reduction Claim: If there is a feasible circulation in the resulting network then it gives a feasible redistribution of papers across the exam rooms. (The solution will actually always exist because we have enough papers and no limits on capacities that stop us from sending papers from one location to another.)

Proof:

Claim 1) If we have a feasible redistribution of papers to exam rooms we will have a feasible circulation in the circulation network constructed:

- Send flow on each edge (in the appropriate direction) equal to the number of papers that are sent out from room to room.
- Send flow from S to each room of R_i
- Send flow from each room to T equal to the final number of papers in that room.
- This will give us a feasible circulation in the network since
 - All lower bound constraints on flow are met (each room got enough papers)
 - All capacity constraints are met (edge capacities were set high enough to allow for movement of any extra papers between locations)
 - All demand conditions are met (especially at T) since redistribution of papers will not cause us to lose or add any flow at any node.

Claim 2) If we have a feasible circulation we can find a feasible redistribution of papers to exam rooms

- Send papers equal to flow sent out from room to room
- This will give us a feasible redistribution of papers since
 - All requirements for papers at exam rooms will be met (since lower bounds on flow from room to T are met)
 - There are no physical limits on how many papers can be transported from room to room

Other proofs follow the same format.

- For candidate solutions 4-7, the claim should say "The max flow value is ____ if and only if ..." and not "max flow exists if and only if..."

-A common mistake is an attempt to have a circulation formulation as in Solution 3 but missing the sink therein, and then incorrectly reducing it to max flow which looks as the one in Solution 5 (circulation -> flow conversion is not necessary, i.e., has no credit). (The penalty

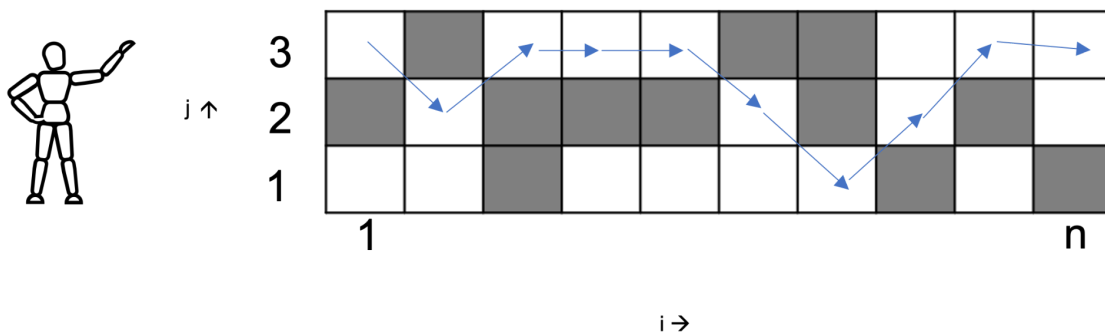
for this error will be reduced from 3.5 to 2.5 IF the claim/final answer is correctly given in terms of the max flow value of the resultant network).

Rubrics 3b:

- **If** the claim is correct,
 - 3 pts: Proof of the Forward claim.
 - 3 pts: Proof of the backward claim.
- Partial credit of at most 2 in total if the claim itself is incorrect (for the proposed solution), but can qualify as a fair attempt..

4) 20 pts

Jack has gotten himself involved in a very dangerous game called the octopus game where he needs to pass a bridge which has some unreliable sections. The bridge consists of $3n$ tiles as shown below. Some tiles are strong and can withstand Jack's weight, but some tiles are weak and will break if Jack lands on them. Jack has no clue which tiles are strong or weak but we have been given that information in an array called $\text{BadTile}(3,n)$ where **$\text{BadTile}(j, i) = 1$ if the tile is weak and 0 if the tile is strong.** At any step Jack can move either to the tile right in front of him (i.e. from tile (j, i) to $(j, i+1)$), or diagonally to the left or right (if they exist). (No sideways or backward moves are allowed and one cannot go from $(1,i)$ to $(3, i+1)$ or from $(3, i)$ to $(1, i+1)$). Using dynamic programming find out how many ways (if any) there are for Jack to pass this deadly bridge. Figure below shows bad tiles in gray and one of the possible ways for Jack to safely cross the bridge alive.



a) Define (in plain English) subproblems to be solved. (4 pts)

$\text{OPT}(j, i)$ = The number of ways Jack can reach the block (j, i) safely from the bridge start

OR

$\text{OPT}(j, i)$ = The subproblems are the number of ways Jack can reach the end safely starting from block (j, i)

Rubrics:

Missing from block or to block : -1

b) Write a recurrence relation for the subproblems (6 pts)

Variant 1:

$$\begin{aligned} \text{OPT}(j, i) &= 0 && \text{if } \text{BadTile}(j, i) = 1; \\ &= \text{OPT}(j, i-1) + \text{OPT}(j+1, i-1) && \text{if } j = 1; \\ &= \text{OPT}(j, i-1) + \text{OPT}(j+1, i-1) + \text{OPT}(j-1, i-1) && \text{if } j = 2; \\ &= \text{OPT}(j, i-1) + \text{OPT}(j-1, i-1) && \text{if } j = 3; \end{aligned}$$

Variant 2 :

In the second variant, $i-1$ is replaced by $i+1$ in each of the terms on the RHS

Variant 3 :

```
OPT1[i] = 0                                if BadTile = 1
        = 1                                if(i ==1)
        = opt2[i-1] + opt1[i-1]           otherwise
```

```
OPT2[i] = 0                                if BadTile = 1
        = 1                                if(i ==1)
        = opt2[i-1] + opt1[i-1] +opt3[i-1] otherwise
```

```
OPT3[i] = 0                                if BadTile = 1
        = 1                                if(i ==1)
        = opt2[i-1] +opt3[i-1]           otherwise
```

Rubrics:

If only j=2 case is written and other cases are not mentioned : -2 (-1 for each case missed)

If recurrence relation is partially correct : -4

Partially wrong recurrence relation -> adding +1 at each step : -2

c) Using the recurrence formula in part b, write pseudocode using iteration to compute the total number of ways to safely cross the bridge. (5 pts)

Make sure you specify

- base cases and their values (2 pts)

- where the final answer can be found (e.g. opt(n), or opt(0,n), etc.) (1 pt)

For Variant 1:

Given BadTile (3,n)

Initialize OPT(3,n) with 0 for each element

OPT(1,1) = 1 if BadTile (1,1) equals to 0, else 0

OPT(2,1) = 1 if BadTile (2,1) equals to 0, else 0

OPT(3,1) = 1 if BadTile (3,1) equals to 0, else 0

for i in 2, 3,4, ..., n:

for j in 1, 2, 3:

if BadTile(j, i) equals to 1:

OPT(j, i) = 0

if j equals to 1:

OPT(j, i) = OPT(j, i-1) + OPT(j+1, i-1)

if j equals to 2:

OPT(j, i) = OPT(j,i-1) + OPT(j+1,i-1) + OPT (j-1,i-1)

if j equals to 3:

OPT(j, i) = OPT(j,i-1) + OPT (j-1,i-1)

return OPT(1, n)+OPT(2, n)+OPT(3, n)

For Variant 2:

Initialize OPT(5,n)

$OPT(0,i) = 0$ //outside the grid

$OPT(4,i) = 0$ // outside the grid

For i in 1 to n:

 For j =1 to 3:

$OPT(j, i) = OPT(j,i-1) + OPT(j+1,i-1) + OPT(j-1,i-1)$

return $OPT(1, n)+OPT(2, n)+OPT(3, n)$

If variant 2 is written recurrence would be just $j = 2$ case

For Variant 3:

Initialize 3 arrays $opt1[], opt2[], opt3[]$;

For i in 2 to n:

 Above recurrence

return $OPT(1, n)+OPT(2, n)+OPT(3, n)$

If variant 2 is written recurrence would be just $j = 2$ case

Rubric:

If where final answer can be found is not mentioned : -1

If recurrence relation is not shown or is wrong : -2

If 3 cases are not shown : -2

If base condition are given according to example : -1

d) What is the complexity of your solution? (1 pt)

Is this an efficient solution? (1 pt)

$O(N)$. Yes.

5) 20 pts

Assume a truck with capacity W is loading. There are n packages with different weights, i.e. $[w_1, w_2, \dots, w_n]$, and all the weights are integers. The company's rule requires that the truck needs to take packages with exactly weight W to maximize profit, but the workers like to save their energies for after work activities and want to load as few packages as possible. Assuming that there are combinations of packages that add up to weight W , design an algorithm to find out the minimum number of packages the workers need to load.

NOTE: For those who assumed n packages to mean “ n types of packages”, we’ve still decided to award points for it out of a maximum possible 13 points. (i.e. for any errors in the attempted solution under this mis-interpretation will have reductions out of the 13 max possible score). This variant of the problem is in fact the one asked in the online version of the exam, please refer to it for solutions and rubrics.

Solution 1:

a) Define (in plain English) subproblems to be solved. (4 pts)

$OPT(w, k) = \text{min packages needed to make up a capacity of exactly } w, \text{ by considering only the first } k \text{ packages}$

OR

Same except, considering packages k onwards up to n (add details below)

b) Write a recurrence relation for the subproblems (6 pts)

If $w \geq w_k$

$OPT(w, k) = \min \{ 1 + OPT(w - w_k, k - 1), OPT(w, k - 1) \}$

Else

$OPT(w, k) = OPT(w, k - 1)$

(No penalty for missing the latter case IF the base cases in ‘c’) include $w < 0$.)

c) Using the recurrence formula in part b, write pseudocode using iteration to compute the minimum number of packages to meet the objective. (5 pts)

1. Initialize for base cases (mentioned below for clarity)

2. For $w = 0$ to W
 For $k = 0$ to n //can switch the inner-outer loops
 If not base case: call recurrence
3. Return ans (mentioned below for clarity)

Make sure you specify

- base cases and their values (2 pts)

$OPT(w,0) = \text{inf}$ for all $w < 0$ (up to $-W$)

$OPT(0,0) = 0$

$OPT(w,k) = \text{infinity}$ for $w < 0$ and all k . (Not required if the recurrence has the second case to ensure w never takes negative values)

- where the final answer can be found (e.g. $\text{opt}(n)$, or $\text{opt}(0,n)$, etc.) (1 pt)

$OPT[W, n]$ in the first definition

$OPT[W, 1]$ in the alternate one

d) What is the complexity of your solution? (1 pt)

Is this an efficient solution? (1 pt)

$O(nW)$ pseudo-polynomial run time

This is not an efficient solution

Solution 2:

a) Define (in plain English) subproblems to be solved. (4 pts)

$OPT(w,k) = \text{min packages needed to make up a capacity of exactly } w, \text{ by considering only the first } k \text{ packages, AND selecting package } k \text{ for sure.}$

b) Write a recurrence relation for the subproblems (6 pts)

The recurrence captures all cases of j where j was the highest index used before the k^{th} one:

If $w \geq w_k$

$OPT(w,k) = 1 + \min_{\{j=0 \text{ to } k-1\}} OPT(w - w_k, j)$

Else

$OPT(w,k) = \text{infinity}$

(No penalty for missing the latter case IF the base cases in 'c)' include $w < 0$.)

c) Using the recurrence formula in part b, write pseudocode using iteration to compute the minimum number of packages to meet the objective. (5 pts)

4. Initialize for base cases (mentioned below for clarity). Initialize rest to infinity

5. For $w = 0$ to W
 - For $k = 0$ to n //can switch the inner-outer loops
 - If (not base case): //call recurrence
 - For $j = 0$ to $k-1$
 - $OPT(w,k) = \min\{OPT(w,k), 1 + OPT(w - w_k, j)\}$
6. Return ans (mentioned below for clarity)

Make sure you specify

- base cases and their values (2 pts)

$OPT(w,0) = \text{inf for all } w > 0$

$OPT(0,0) = 0$

$OPT(w,k) = \text{infinity for } w < 0 \text{ and all } k.$ (Not required if the recurrence has the second case to ensure w never takes negative values)

- where the final answer can be found (e.g. $\text{opt}(n)$, or $\text{opt}(0,n)$, etc.) (1 pt)

$\text{Max}_k OPT(W, k)$

d) What is the complexity of your solution? (1 pt)

Is this an efficient solution? (1 pt)

$O(n^2W)$ pseudo-polynomial run time

This is not an efficient solution